



Naan Mudhalvan

Industrial IoT and Industry 4.0

Tektork Private Limited, Coimbatore

MQTT

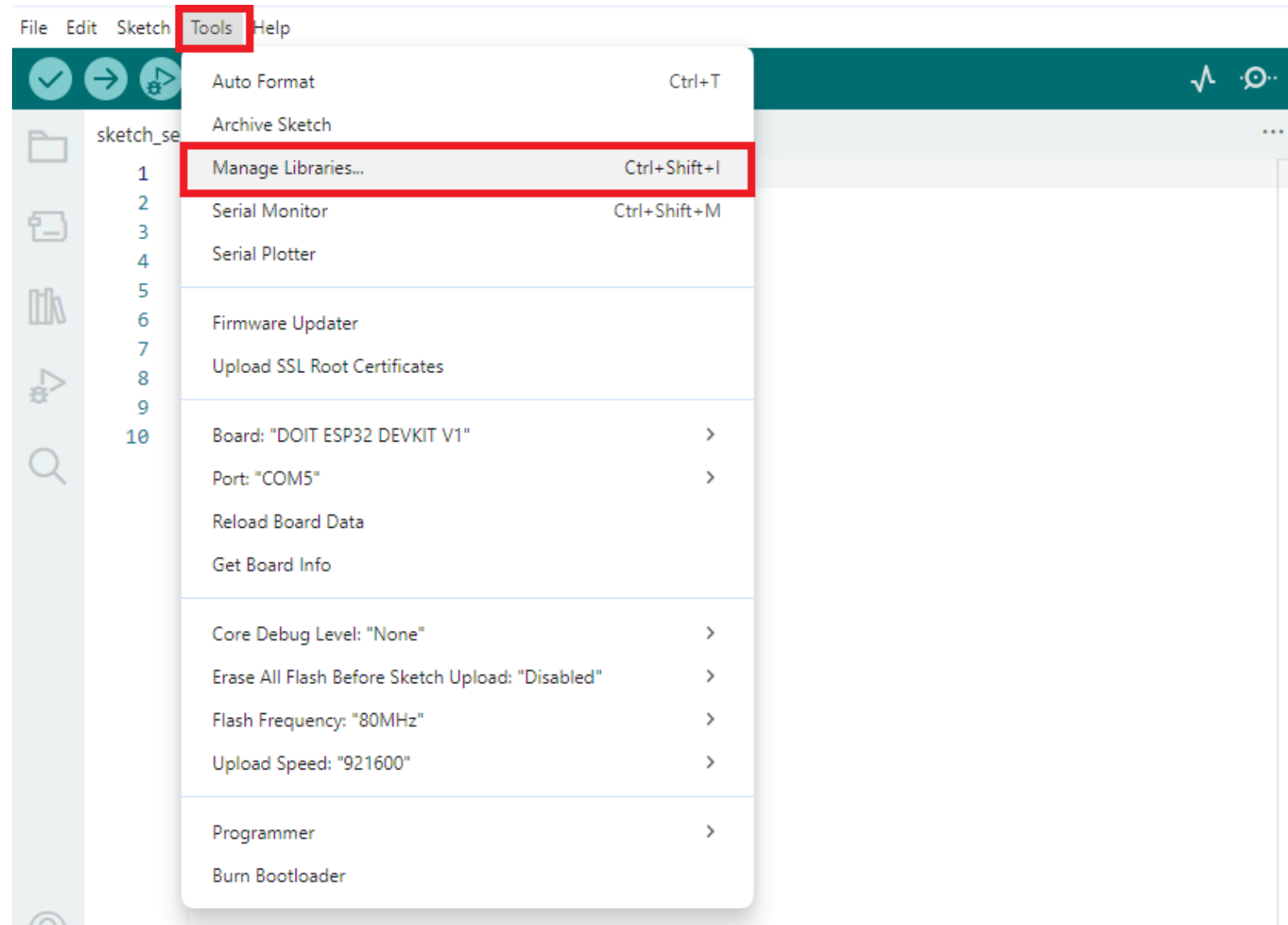
Publishing and Receiving

Arduino Code

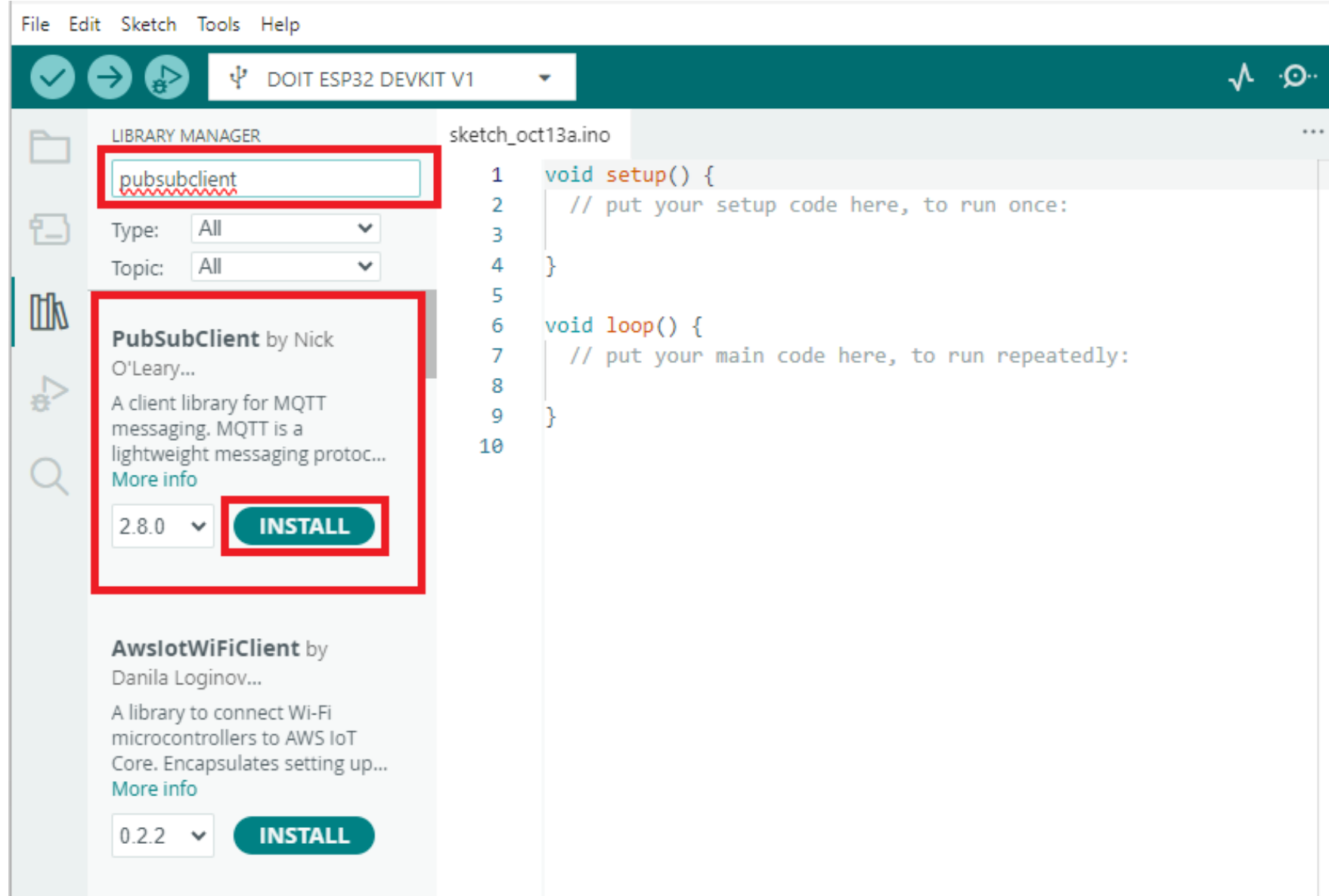
MQTT Receiving Arduino Code

Library Installation - PubSubClient (If required, install the PubSubClient library)

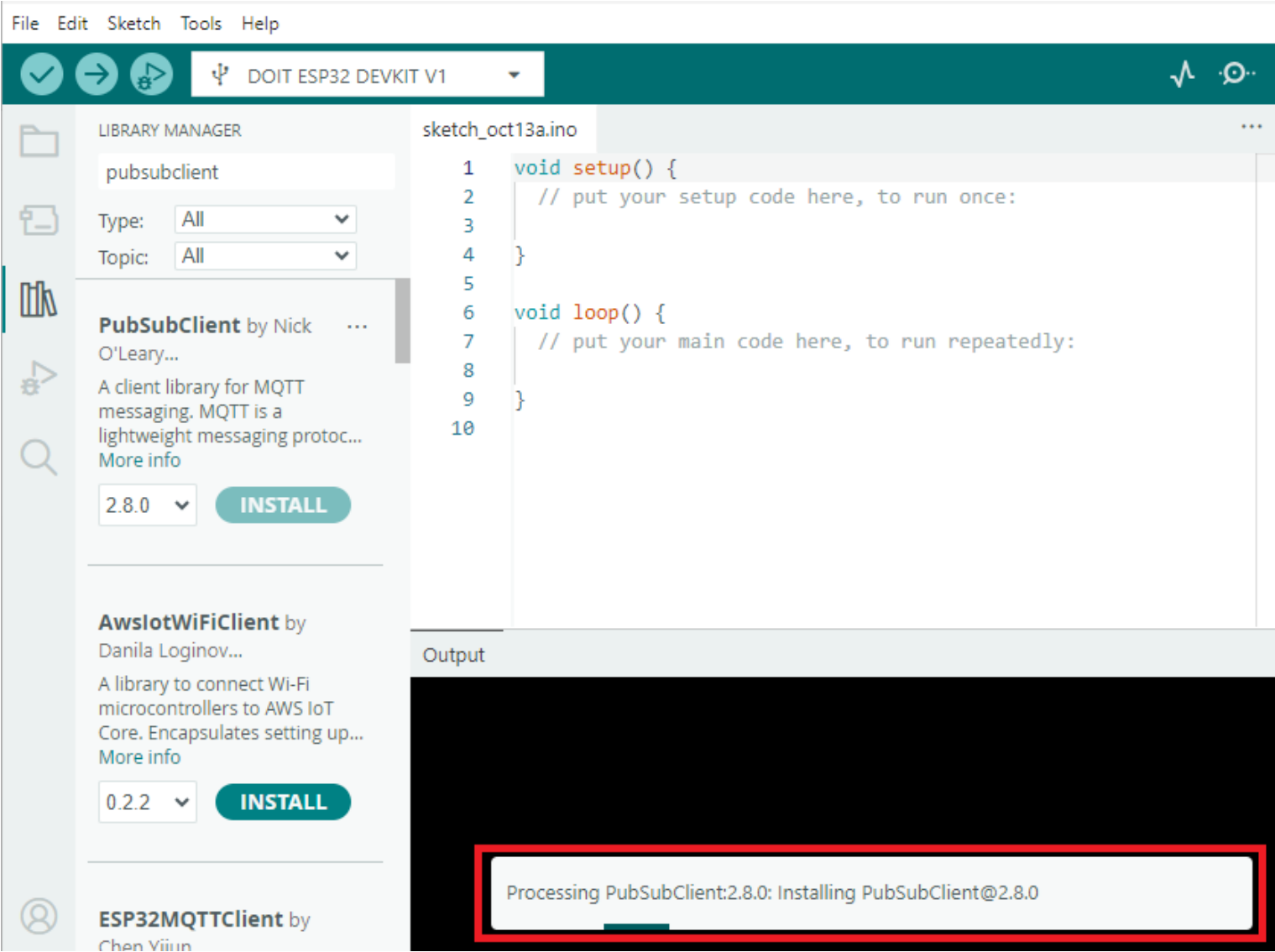
- In the Arduino IDE, go to the Tools menu, then select Manage Libraries... to open the Library Manager.



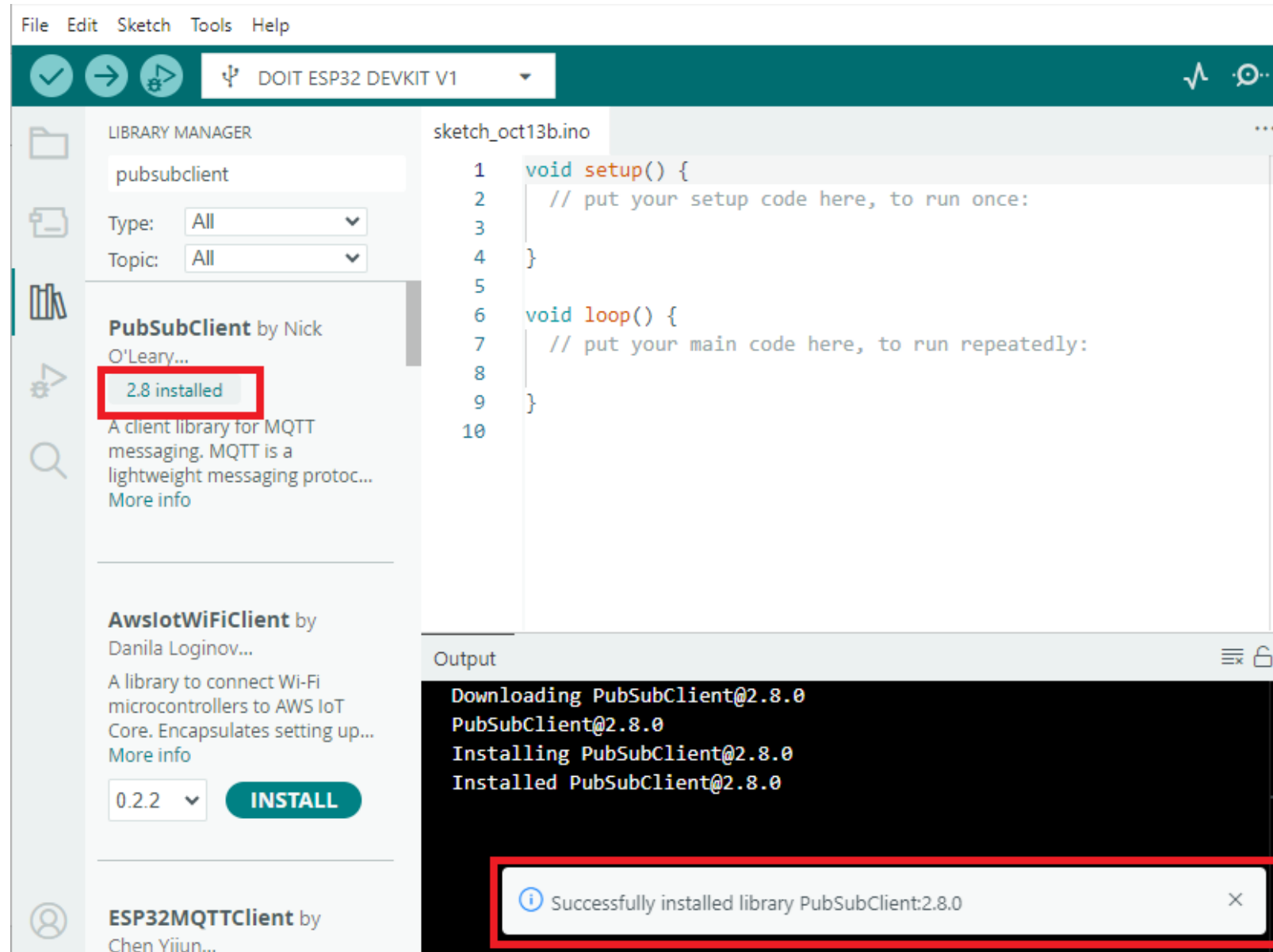
- In the Library Manager search box, type **PubSubClient (by Nick O'Leary)** , then select the **PubSubClientLibrary** and click Install



- The PubSubClient library installation process is in progress. Wait until the installation completes; once finished, you can include the library in your Arduino sketch and start using the MQTT protocol



- After the installation is complete, It shows Installed.



LED Controller MQTT Code

```
#include <WiFi.h>                // WiFi library for ESP32
#include <PubSubClient.h>         // MQTT client library

// WiFi credentials (SSID and Password)
const char* ssid = "Pradeep";
const char* password = "123456789";

// Public MQTT broker info (HiveMQ broker)
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;      // MQTT default port
const char* mqtt_topic_led1 = "esp32/led1"; // Topic for LED1 control
const char* mqtt_topic_led2 = "esp32/led2"; // Topic for LED2 control

WiFiClient espClient;           // Create WiFi client
PubSubClient client(espClient); // Create MQTT client using WiFi

// Define LED GPIO pins
#define LED1 33 // LED1 connected to GPIO33
#define LED2 4  // LED2 connected to GPIO4
```

```
// Function to connect ESP32 to WiFi
void setup_wifi() {
    delay(10);
    Serial.println();
    Serial.print("Connecting to WiFi: ");
    Serial.println(ssid);

    WiFi.begin(ssid, password); // Start WiFi connection
    while (WiFi.status() != WL_CONNECTED) { // Wait until connected
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected!"); // Success message
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP()); // Print IP assigned to ESP32
}

// MQTT callback function to handle incoming messages
void callback(char* topic, byte* payload, unsigned int length) {
    payload[length] = '\0'; // Null-terminate payload for String conversion
    String message = String((char*)payload); // Convert payload to String
    Serial.print("Message arrived [");
    Serial.print(topic);
    Serial.print("]: ");
    Serial.println(message);
}
```

```
// Control LED1 based on MQTT message
if (String(topic) == mqtt_topic_led1)
{
    if (message == "ON")    digitalWrite(LED1, HIGH); // Turn LED1 ON
    else if (message == "OFF") digitalWrite(LED1, LOW); // Turn LED1 OFF
}

// Control LED2 based on MQTT message
if (String(topic) == mqtt_topic_led2)
{
    if (message == "ON")    digitalWrite(LED2, HIGH); // Turn LED2 ON
    else if (message == "OFF") digitalWrite(LED2, LOW); // Turn LED2 OFF
}
}
```

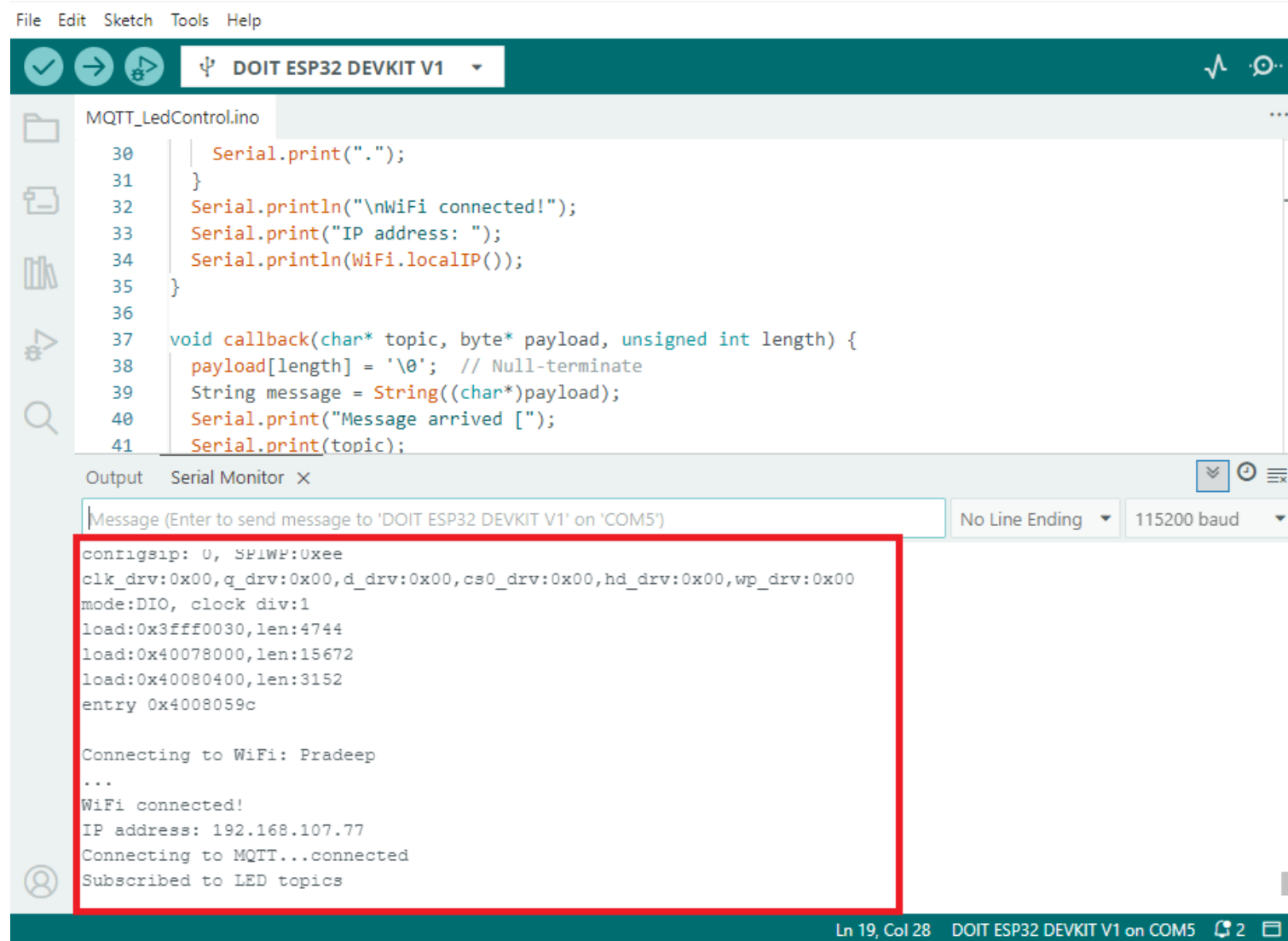
```
// Function to reconnect to MQTT broker if disconnected
void reconnect()
{
    while (!client.connected()) { // Loop until connected
        Serial.print("Connecting to MQTT...");
        // Connect to broker with unique client ID
        if (client.connect("ESP32_LED_Controller"))
        {
            Serial.println("connected");
            // Subscribe to both LED control topics
            client.subscribe(mqtt_topic_led1);
            client.subscribe(mqtt_topic_led2);
            Serial.println("Subscribed to LED topics");
        } else { // If failed, wait and retry
            Serial.print("failed, rc=");
            Serial.print(client.state()); // Print error code
            Serial.println(" retrying in 5s");
            delay(5000);
        }
    }
}
```

```
// Initial setup
void setup()
{
    Serial.begin(115200);           // Start serial monitor
    pinMode(LED1, OUTPUT);          // Set LED1 pin as output
    pinMode(LED2, OUTPUT);          // Set LED2 pin as output
    digitalWrite(LED1, LOW);        // Initially turn OFF LED1
    digitalWrite(LED2, LOW);        // Initially turn OFF LED2

    setup_wifi();                   // Connect to WiFi
    client.setServer(mqtt_server, mqtt_port); // Set MQTT server & port
    client.setCallback(callback);   // Set MQTT callback function
}

// Main loop
void loop()
{
    if (!client.connected()) reconnect(); // Ensure MQTT connection
    client.loop(); // Process incoming messages and keep connection alive
}
```

- After uploading the code to the ESP32, once the upload is completed, press the reset button. The ESP32 will connect to Wi-Fi, print the IP address, and establish a connection with the MQTT broker.



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for checking, running, and uploading code. The selected board is DOIT ESP32 DEVKIT V1. The code editor displays the MQTT_LedControl.ino file with the following code:

```

30   Serial.print(".");
31   }
32   Serial.println("\nWiFi connected!");
33   Serial.print("IP address: ");
34   Serial.println(WiFi.localIP());
35   }
36
37   void callback(char* topic, byte* payload, unsigned int length) {
38       payload[length] = '\0'; // Null-terminate
39       String message = String((char*)payload);
40       Serial.print("Message arrived [");
41       Serial.print(topic);

```

The Serial Monitor is open, showing the output of the code. The output is as follows:

```

config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4744
load:0x40078000,len:15672
load:0x40080400,len:3152
entry 0x4008059c

Connecting to WiFi: Pradeep
...
WiFi connected!
IP address: 192.168.107.77
Connecting to MQTT...connected
Subscribed to LED topics

```

The Serial Monitor settings are set to "No Line Ending" and "115200 baud". The status bar at the bottom indicates "Ln 19, Col 28 DOIT ESP32 DEVKIT V1 on COM5".

Note:

Ensure Network Connectivity

- Make sure your **computer running Node-RED** and the **ESP32** are on the **same WiFi network**.
- Public MQTT broker (e.g., broker.hivemq.com) should be reachable from your network.

Note:

Install Required Node-RED Packages

- Open Node-RED, go to the Menu, select Manage Palette, type node-red-dashboard into the search box, go to the Install tab, and then click Install.
- Install **node-red-dashboard** (for UI buttons and switches).
- MQTT nodes usually come pre-installed; if not, install **node-red-node-mqtt**

Note:

Check MQTT Broker Connectivity.

In Node-RED, go to the MQTT node configuration and make sure of the following:

- **Server:** broker.hivemq.com
- **Port:** 1883
- **Connection status:** shows "**connected**" (in the bottom right corner of the MQTT node).

Note:

Verify Topic Names

Make sure the **topics match exactly** between ESP32 and Node-RED.

In the Arduino code:

```
const char* led1_topic = "esp32/led1";  
const char* led2_topic = "esp32/led2";
```

In Node-RED:

- MQTT Out node for LED1 → topic must be **esp32/led1**
- MQTT Out node for LED2 → topic must be **esp32/led2**

Topic names are **case-sensitive**. (ESP32/LED1 ≠ esp32/led1)

Note:

Check Payload Values

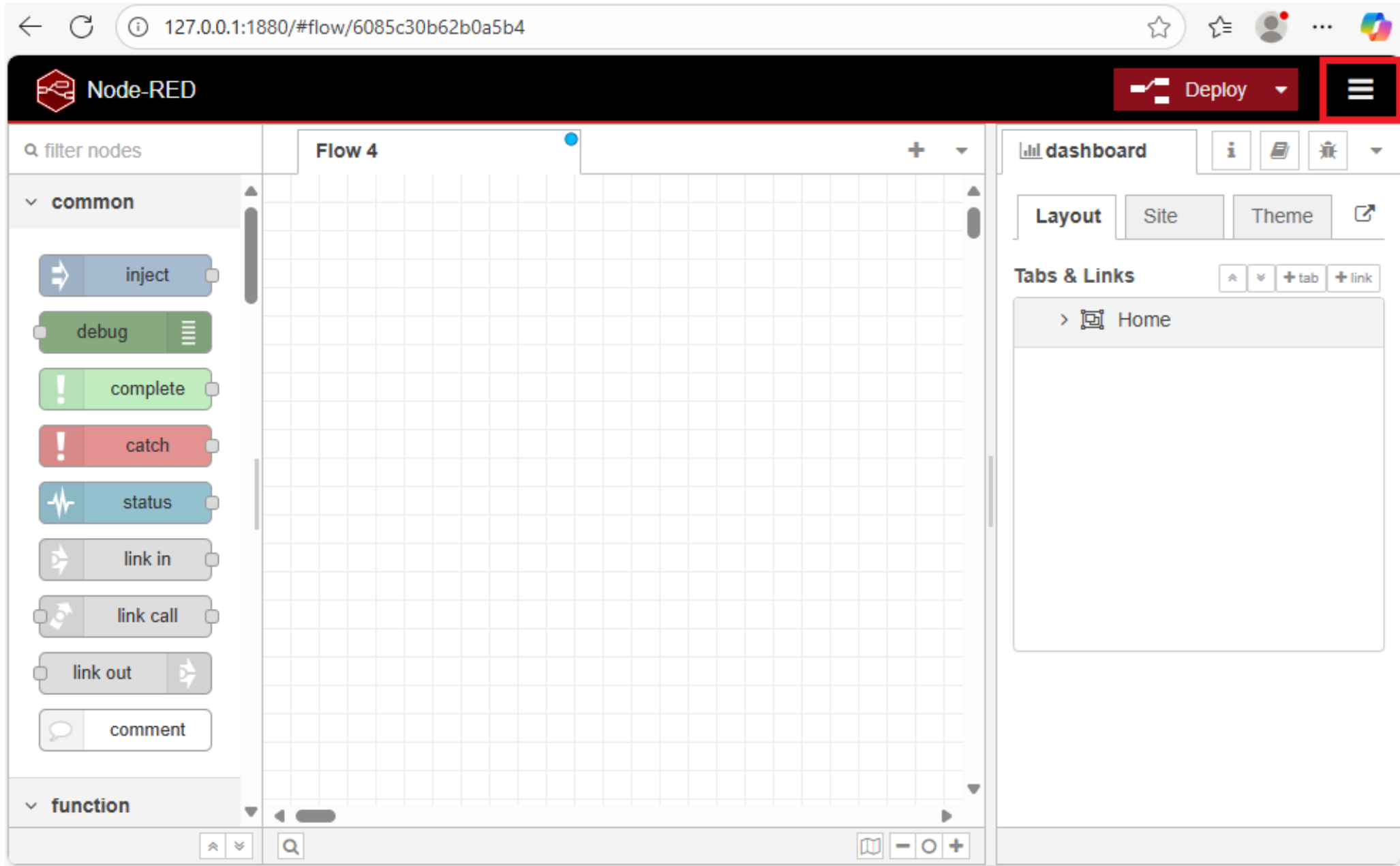
In Node-RED Button node:

- Go to **Edit Button**
- Set **Payload type** change to **String**
- Payload value change to "ON" (without quotes)
- Add another button with payload "OFF"

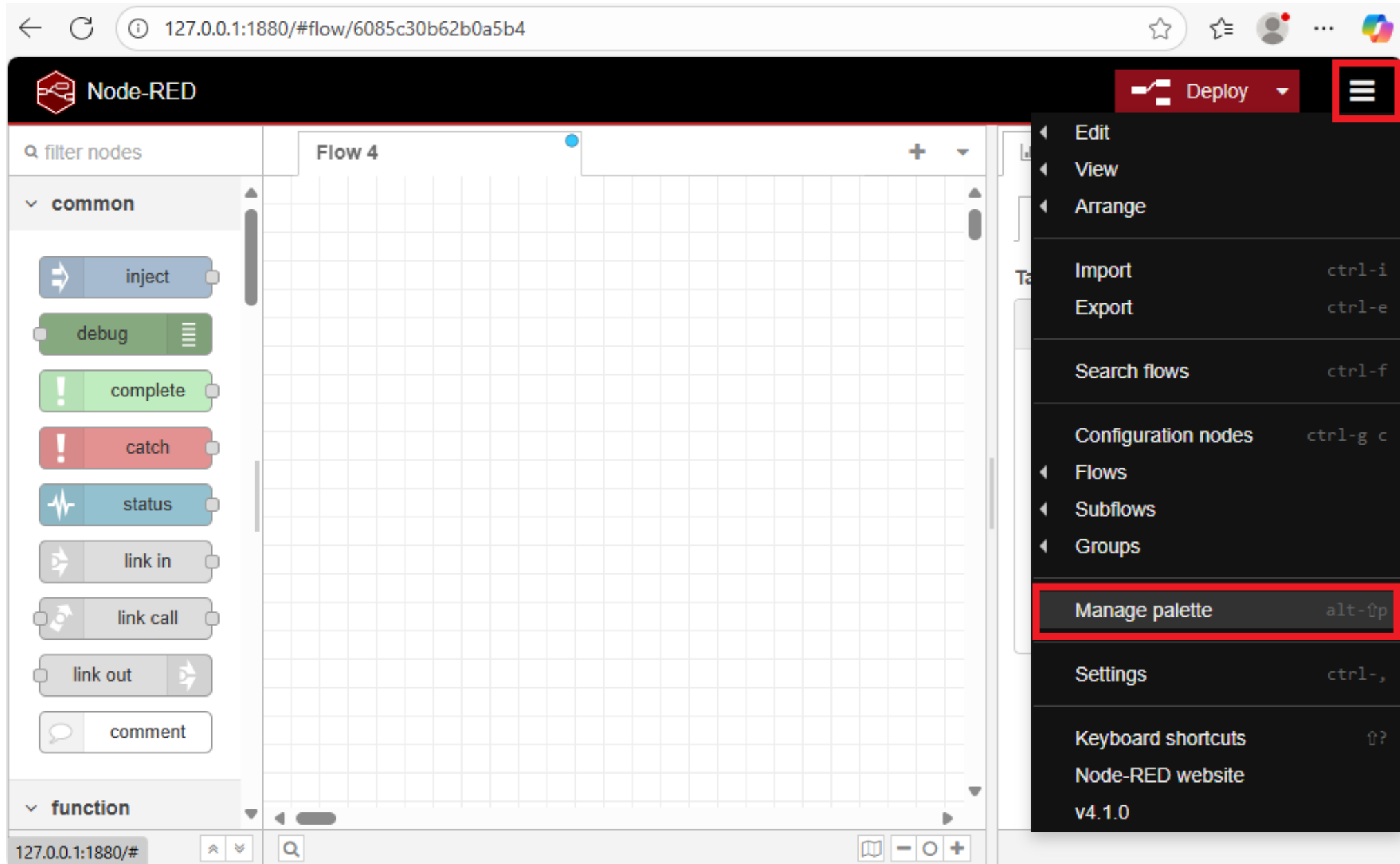
Or use one button with **toggle** mode:

- Enable "Send alternate payloads"
- Payload 1 change to ON
- Payload 2 change to OFF

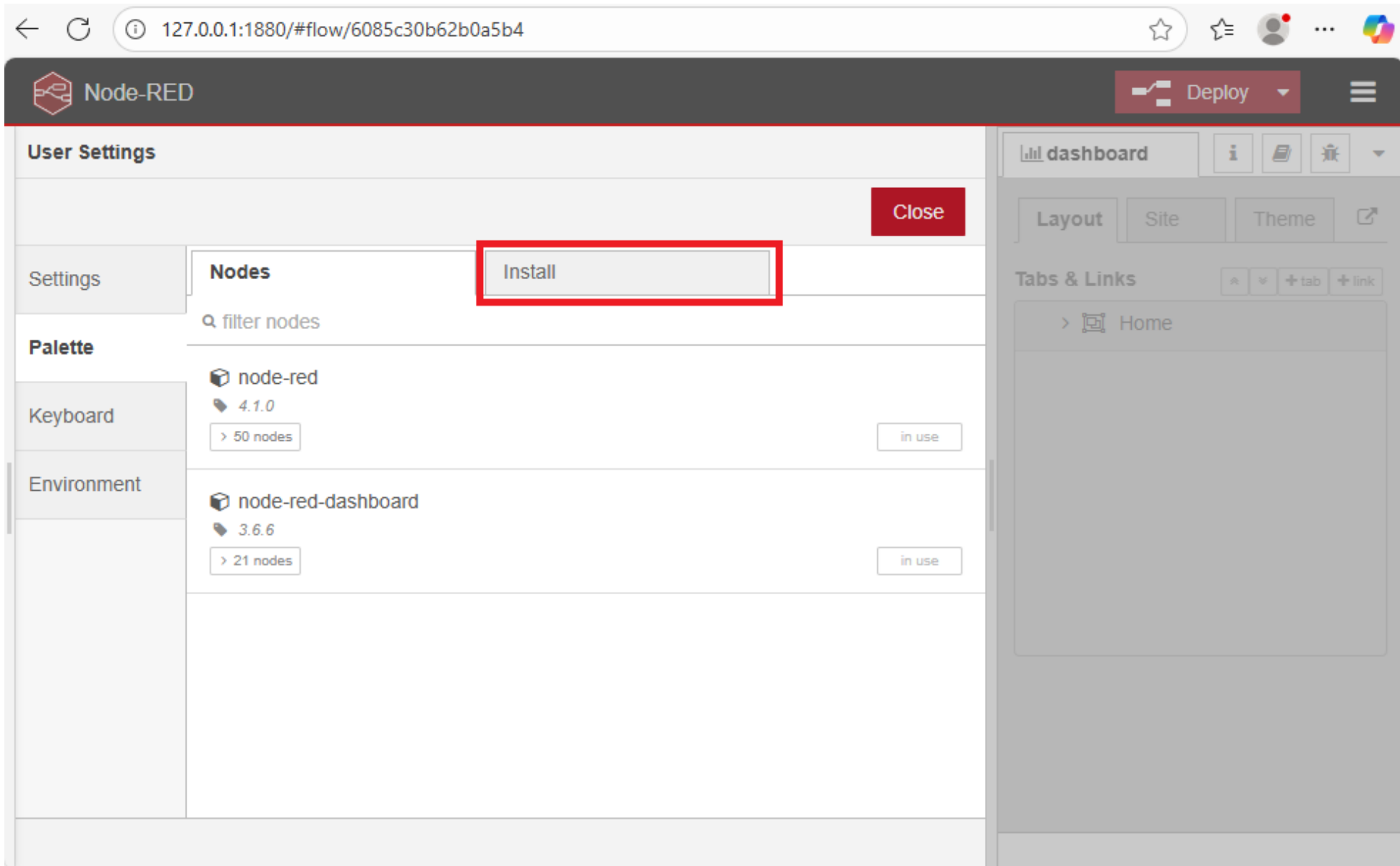
- Open Node-RED in your browser: <http://localhost:1880>



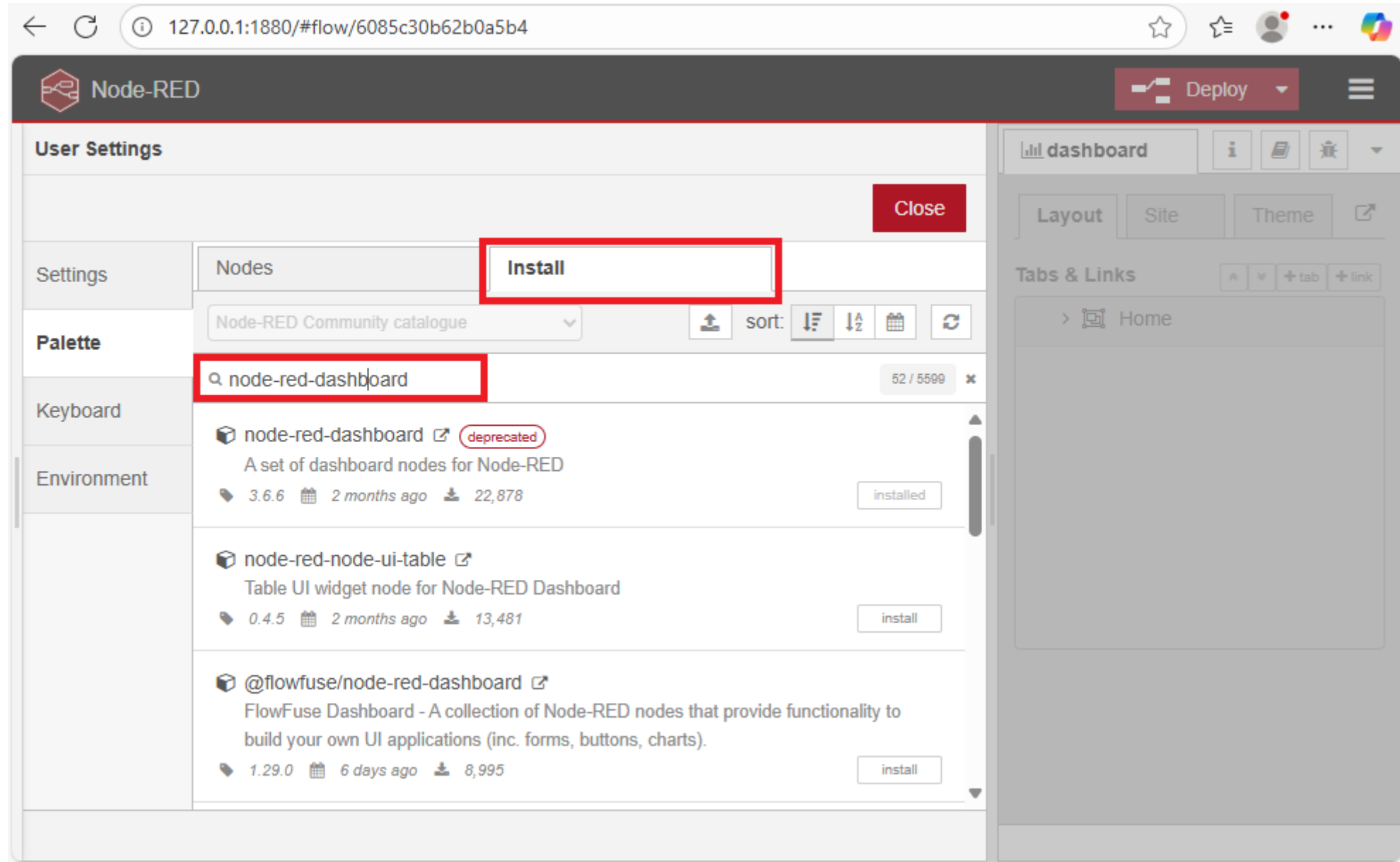
- Click the **menu** () in the top-right corner then select “**Manage palette**”.



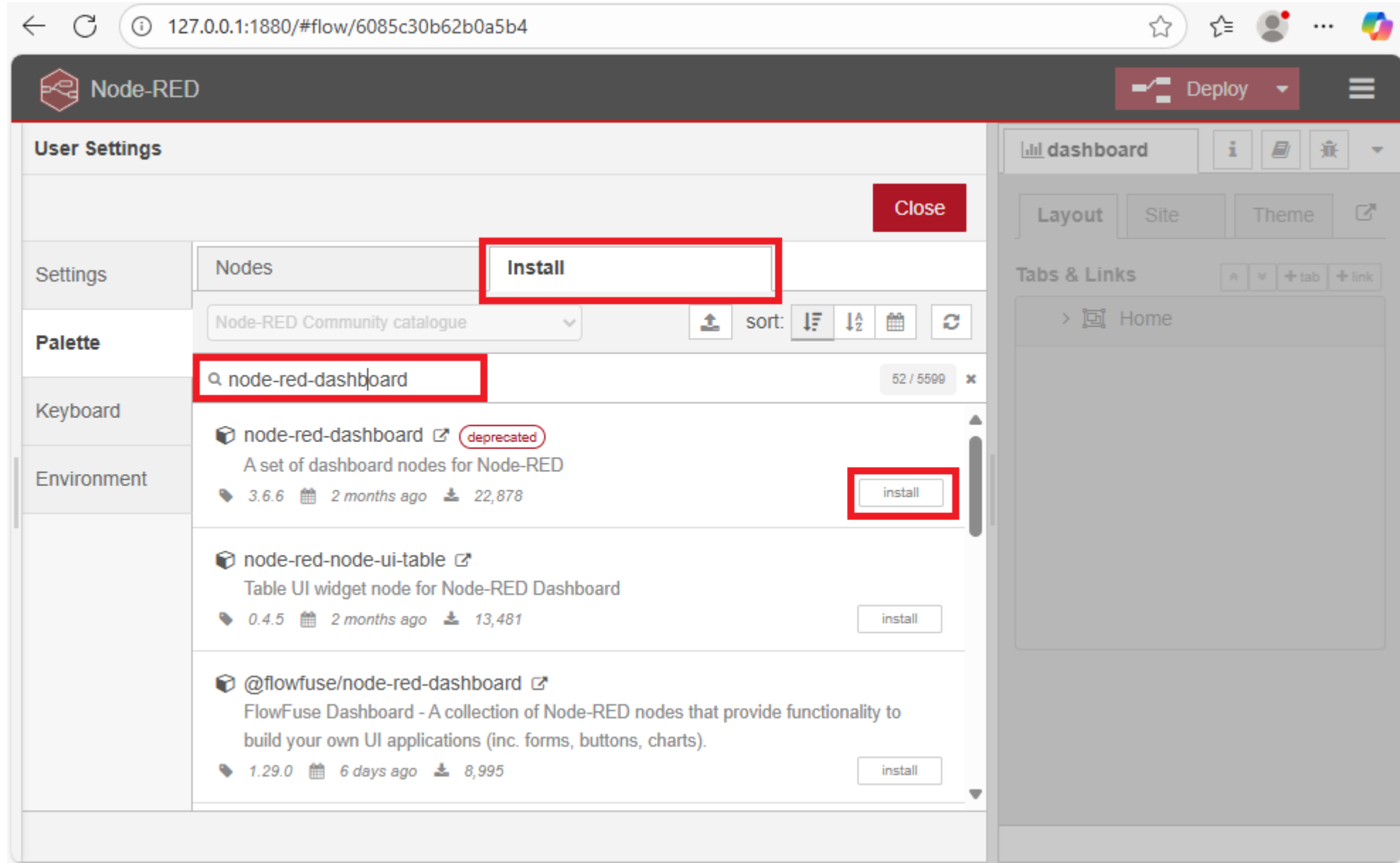
- Go to the “Install” tab.



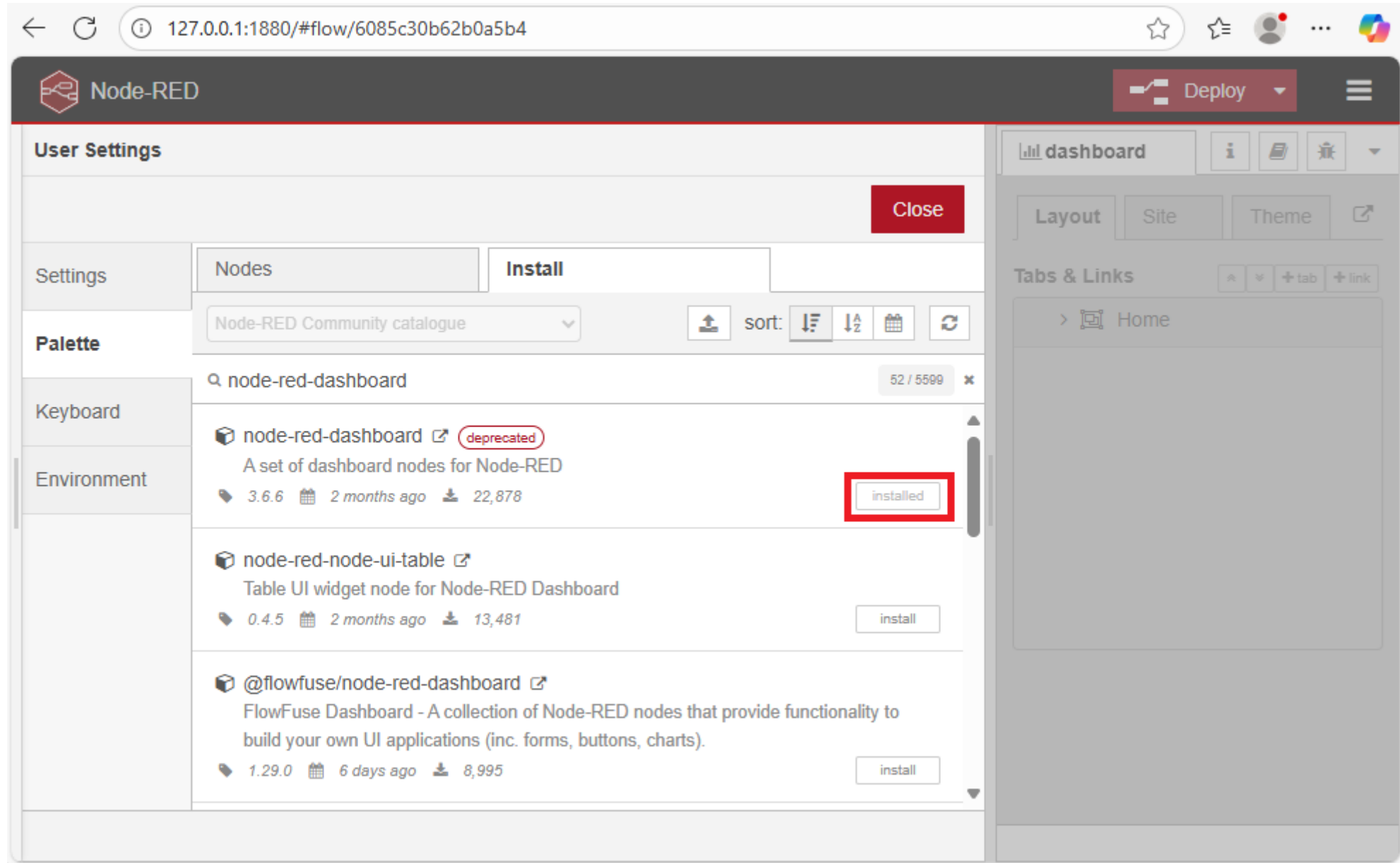
- In the search box, type: node-red-dashboard



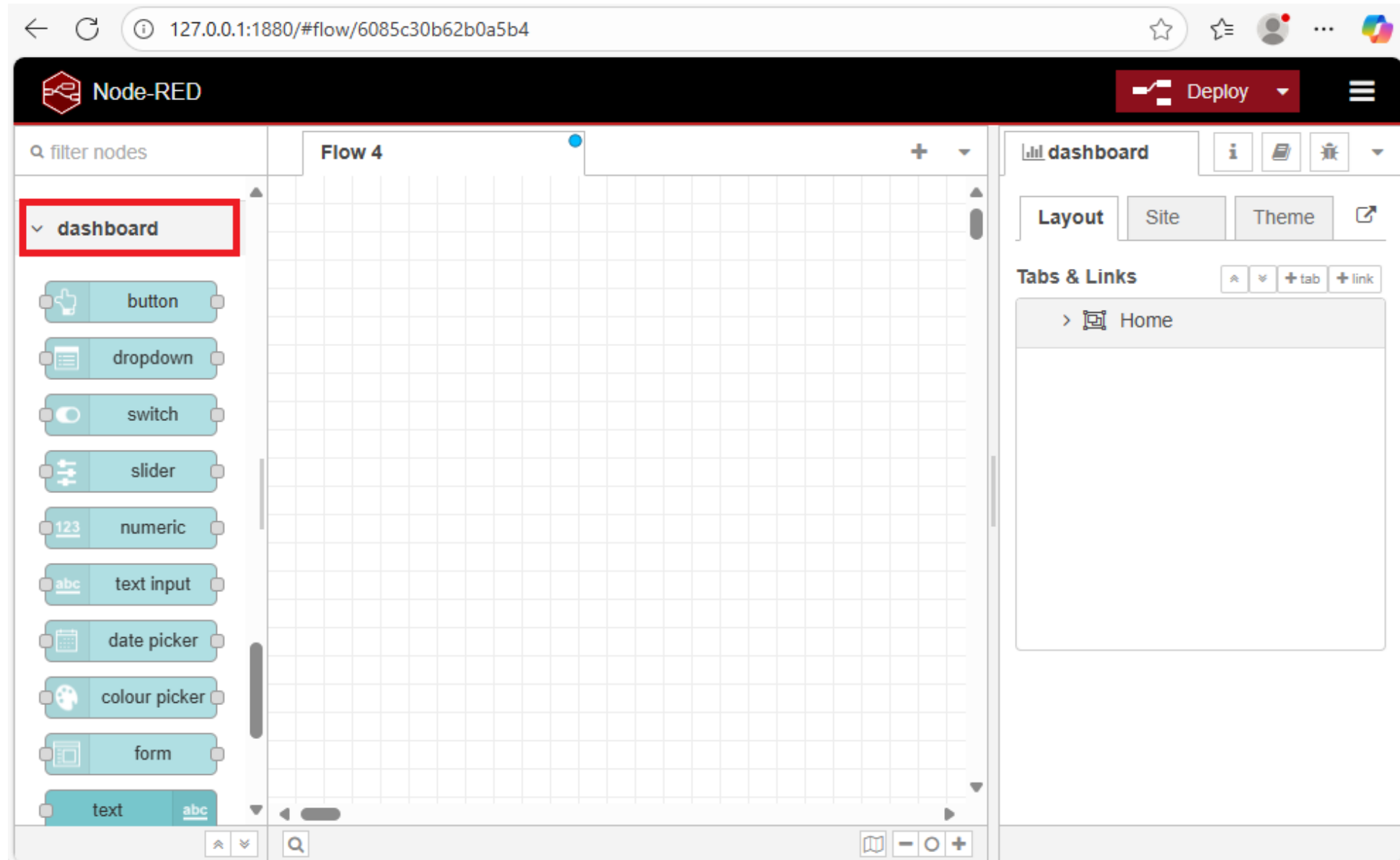
- Click the **Install** button next to **node-red-dashboard** (by Node-RED developers).



- Dashboard is now installed



- Once done, you'll see **"dashboard" nodes** (like ui_switch, ui_button, ui_text, ui_chart, etc.) appear in your **left node palette** under the **Dashboard** category.



Note:

Access the Dashboard

Once installed and your flow includes dashboard nodes:

Open your Node-RED editor:

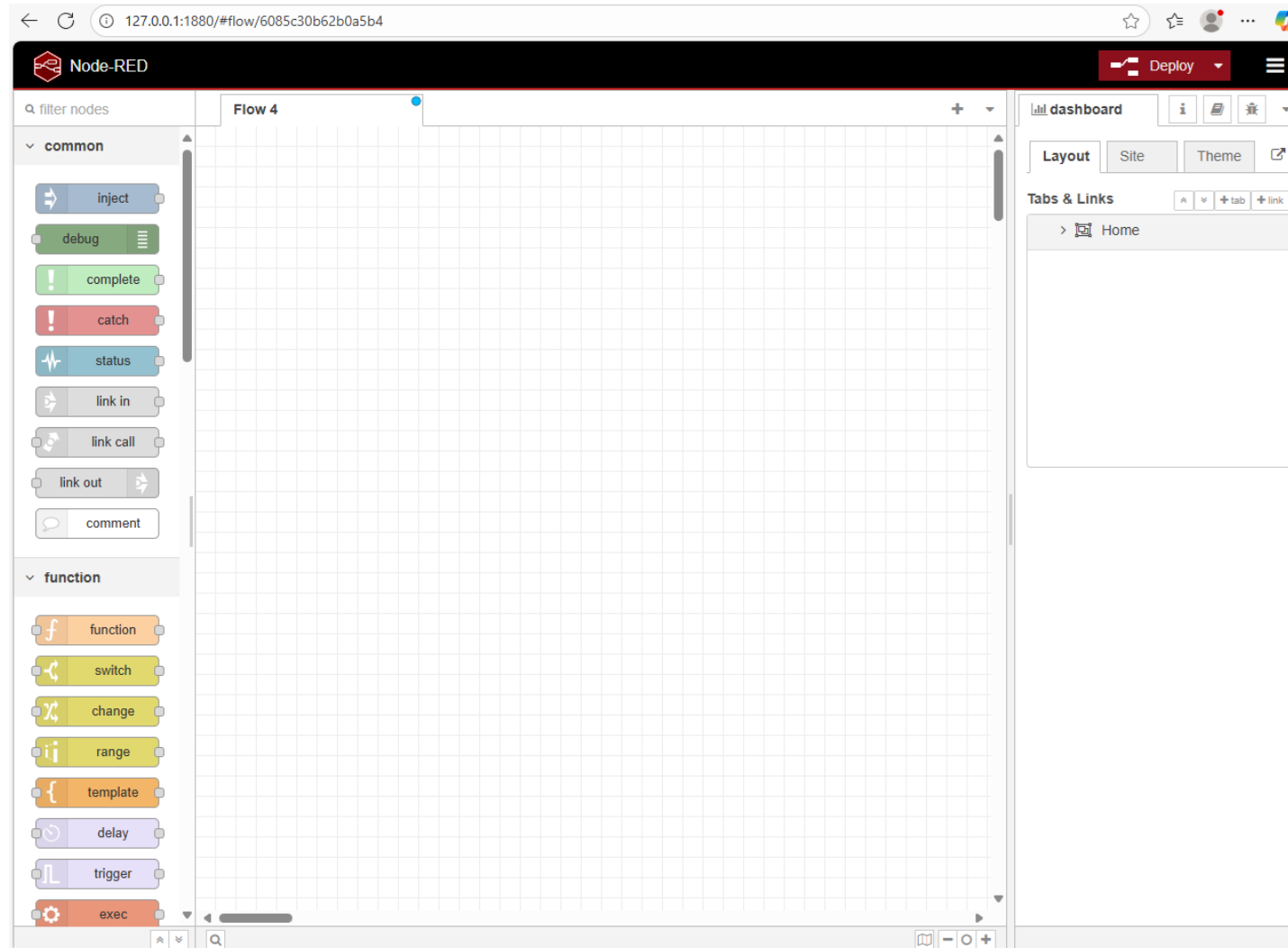
- <http://localhost:1880>

Then open the dashboard:

- <http://localhost:1880/ui>

Open Node-RED editor

- Go to <http://localhost:1880> in your web browser.
- You'll see the **Node-RED workspace** with the **node palette** on the **left side**.



Locate the MQTT nodes

- In the left palette, scroll down until you find the “**network**” category.
- Inside this category, you’ll see the following nodes:
 - **mqtt in** — receives messages from a topic
 - **mqtt out** — publishes messages to a topic

These are what you’ll use to connect Node-RED to your ESP32 (via the HiveMQ broker).

← ↻ 127.0.0.1:1880/#flow/6085c30b62b0a5b4

 Node-RED

Deploy

filter nodes

Flow 4

network

- mqtt in
- mqtt out
- http in
- http response
- http request
- websocket in
- websocket out
- tcp in
- tcp out

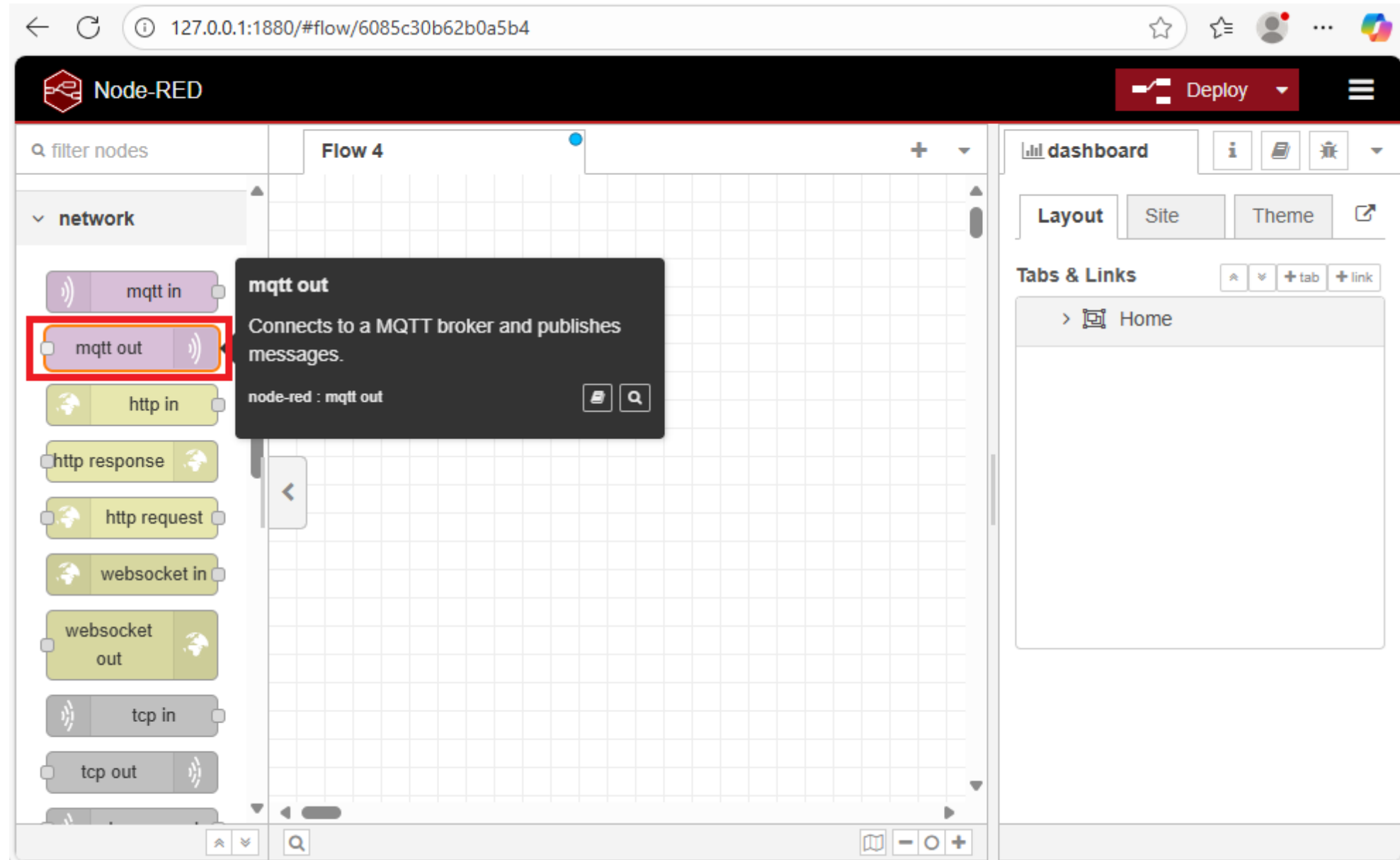
dashboard

Layout Site Theme

Tabs & Links

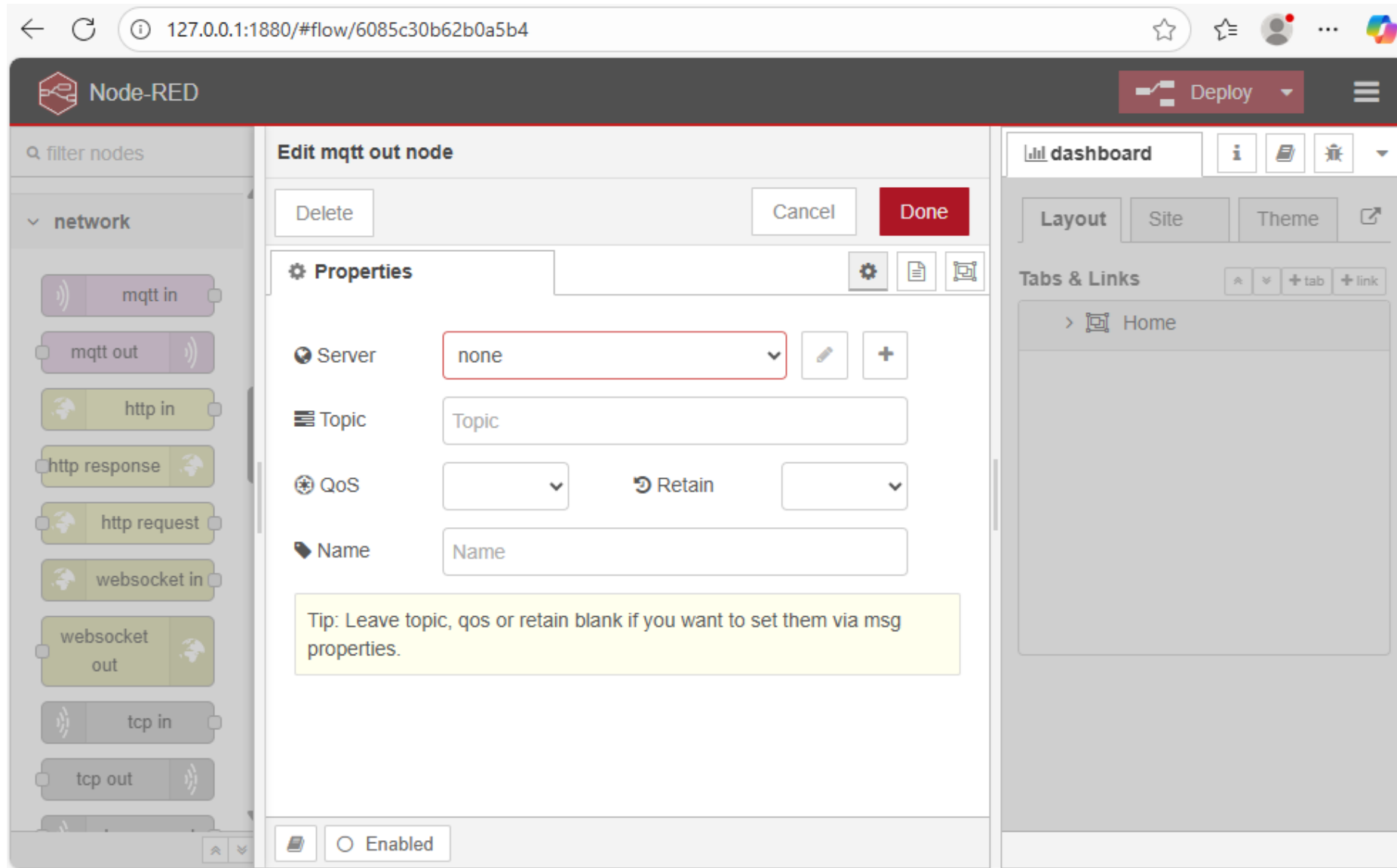
- Home

- Drag the nodes onto your flow area.
- Drag the **mqtt out** node — this node is used to send **ON/OFF commands** to the ESP32.



Configure MQTT connection

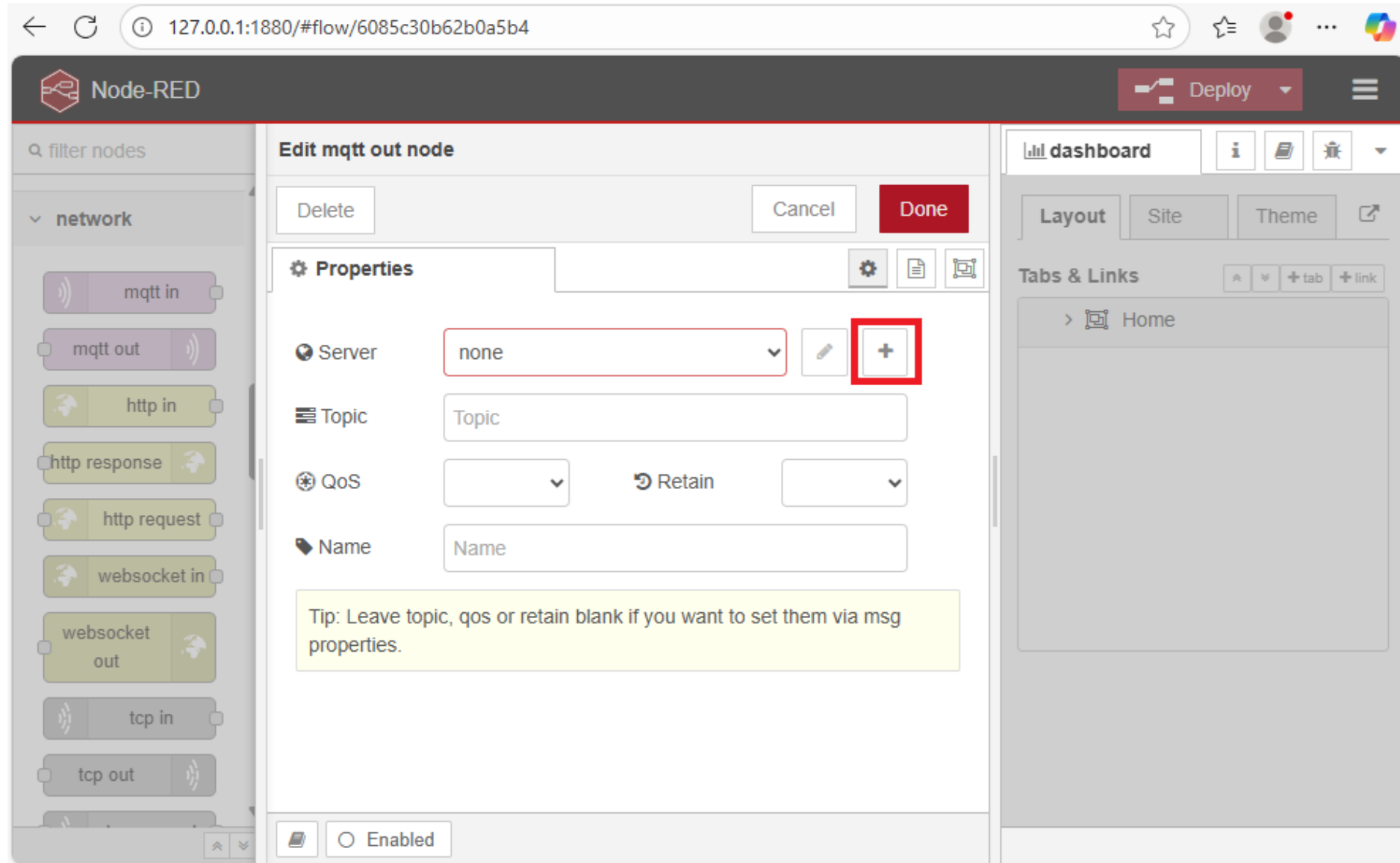
- **Double-click the MQTT node** to open its configuration settings.



The screenshot shows the Node-RED web interface in a browser. The address bar displays the URL `127.0.0.1:1880/#flow/6085c30b62b0a5b4`. The interface is divided into three main panels:

- Left Panel (Nodes):** A sidebar with a search bar labeled "filter nodes". Under the "network" category, various nodes are listed, including "mqtt in", "mqtt out", "http in", "http response", "http request", "websocket in", "websocket out", "tcp in", and "tcp out".
- Center Panel (Edit mqtt out node):** This panel displays the configuration settings for the selected "mqtt out" node. It includes:
 - Buttons: "Delete", "Cancel", and "Done".
 - Properties Section:**
 - Server:** A dropdown menu currently set to "none".
 - Topic:** A text input field containing the placeholder text "Topic".
 - QoS:** A dropdown menu.
 - Retain:** A checkbox and a dropdown menu.
 - Name:** A text input field containing the placeholder text "Name".
 - Tip:** A yellow box with the text: "Tip: Leave topic, qos or retain blank if you want to set them via msg properties."
 - Enabled:** A radio button labeled "Enabled" is selected.
- Right Panel (Dashboard):** A sidebar for the dashboard, showing a "dashboard" tab, "Layout", "Site", and "Theme" settings, and a "Tabs & Links" section with a "Home" link.

- In the **MQTT node configuration settings**, click the **plus (+)** icon to add a new MQTT broker. Then, enter the **Server**, **Port**, and **MQTT topic** details.



127.0.0.1:1880/#flow/6085c30b62b0a5b4

Node-RED

Deploy

Edit mqtt out node > Add new mqtt-broker config node

Cancel Add

Properties

Connection Security Messages

Server Port

☒ Connect automatically

☐ Use TLS

Protocol

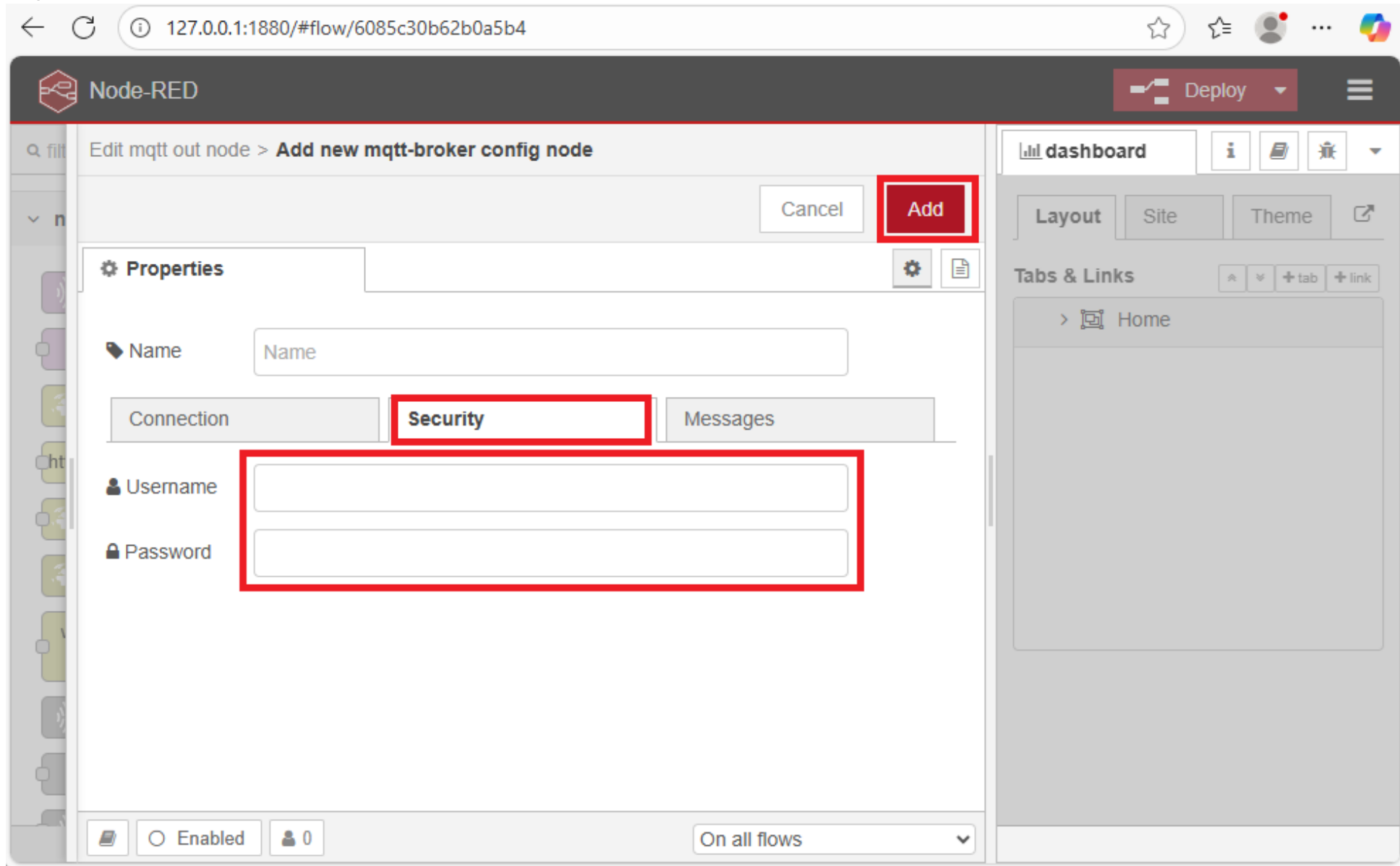
Client ID

Keep Alive

Session ☒ Use clean session

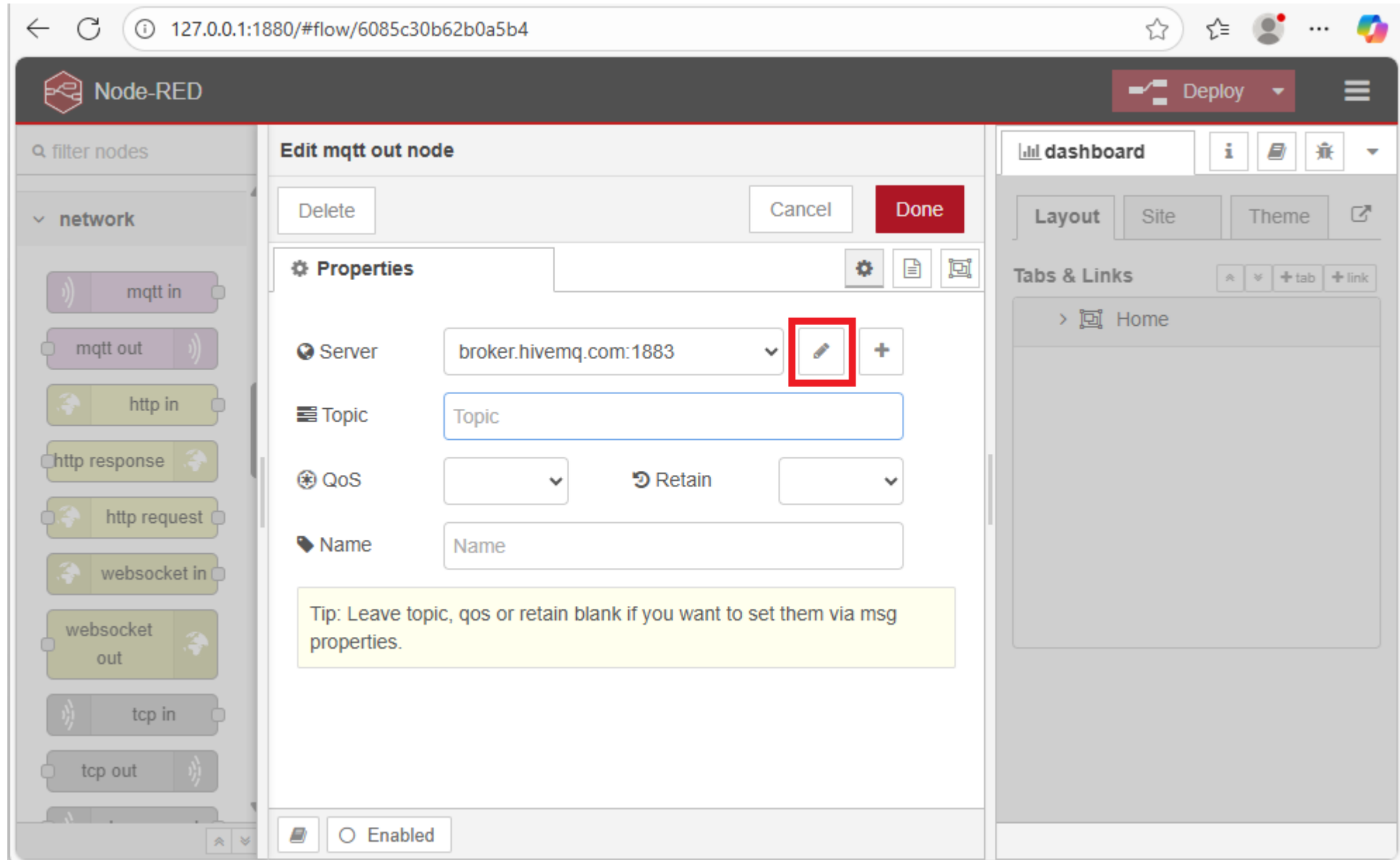
☐ Enabled ☐ Disabled 0 On all flows

- Leave the username and password fields blank (since it's a public broker), then select the **Add** option.

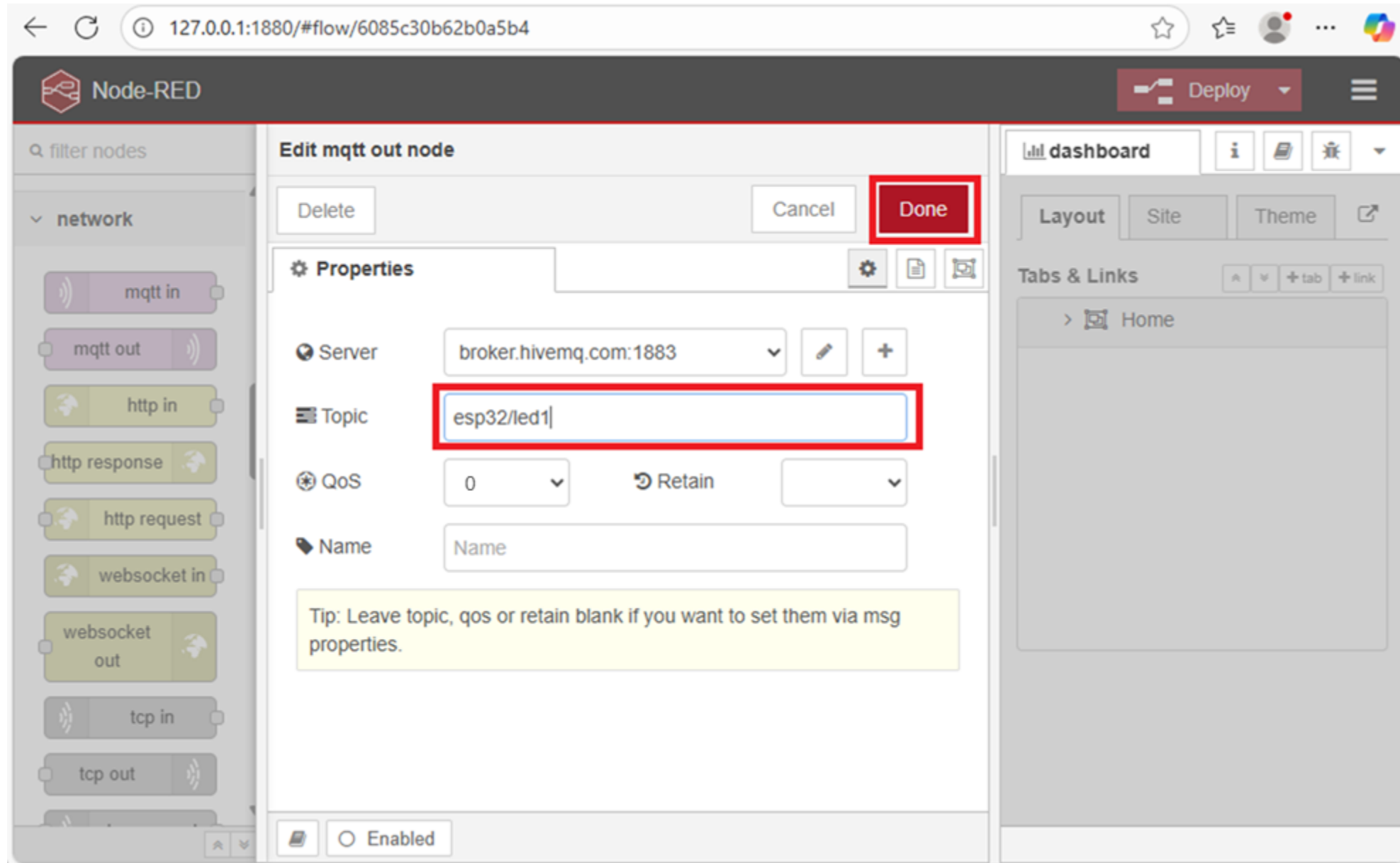


The screenshot shows the Node-RED web interface. The browser address bar displays '127.0.0.1:1880/#flow/6085c30b62b0a5b4'. The Node-RED header includes the logo, the text 'Node-RED', a 'Deploy' button, and a menu icon. The main workspace shows a flow editor with a search filter 'Edit mqtt out node > Add new mqtt-broker config node'. A dialog box is open for adding a new MQTT broker configuration. The dialog has a 'Cancel' button and a red 'Add' button. The 'Properties' section is visible, with a 'Name' field. Below this, there are three tabs: 'Connection', 'Security' (which is selected and highlighted with a red box), and 'Messages'. Under the 'Security' tab, there are two input fields: 'Username' and 'Password', both of which are empty and highlighted with a red box. At the bottom of the dialog, there are checkboxes for 'Enabled' and '0', and a dropdown menu set to 'On all flows'.

- Click the **pencil icon** next to the **Server** field to add a new MQTT broker configuration.

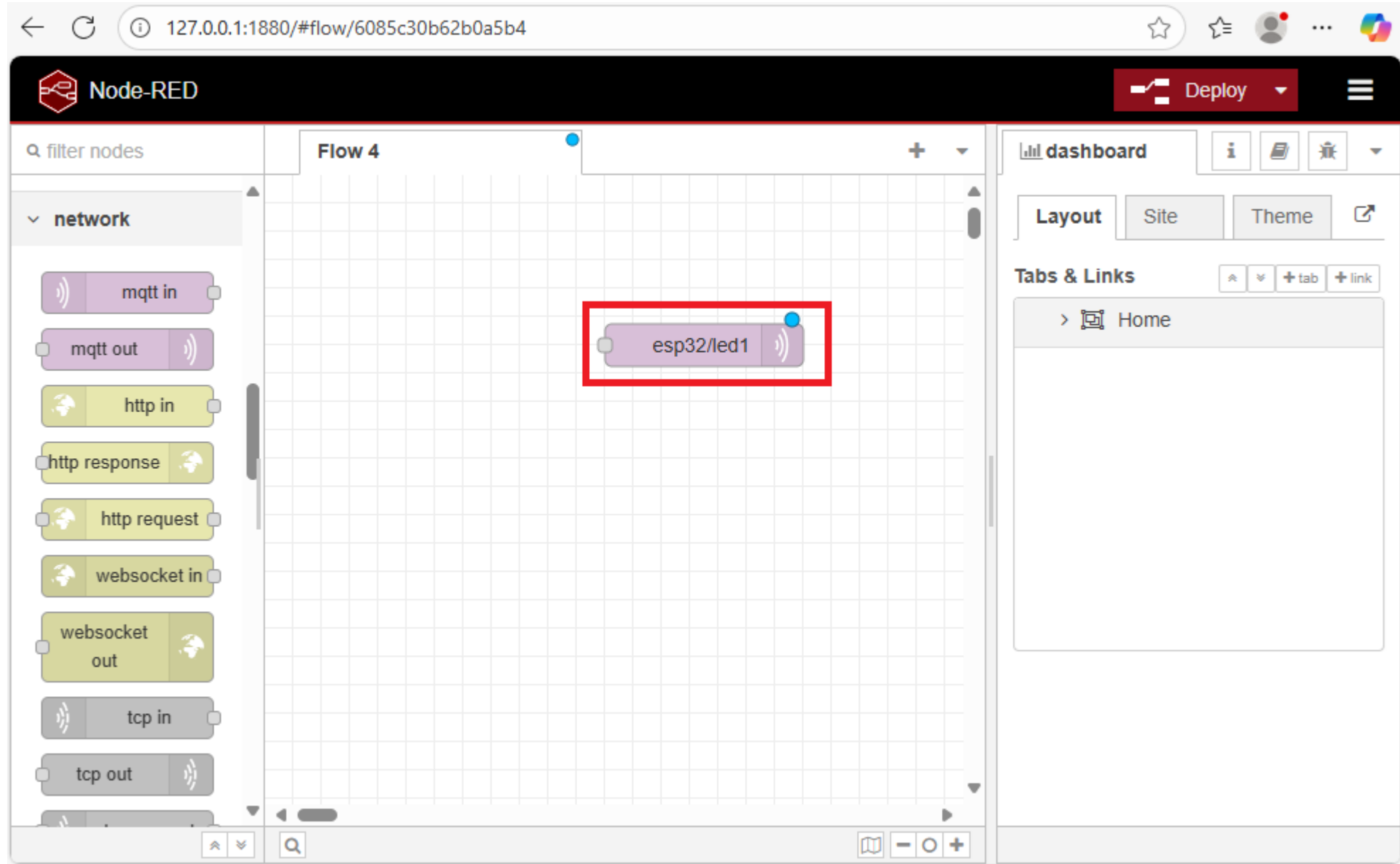


- Next, add the topic "**esp32/led1**", then select the **Done** option.

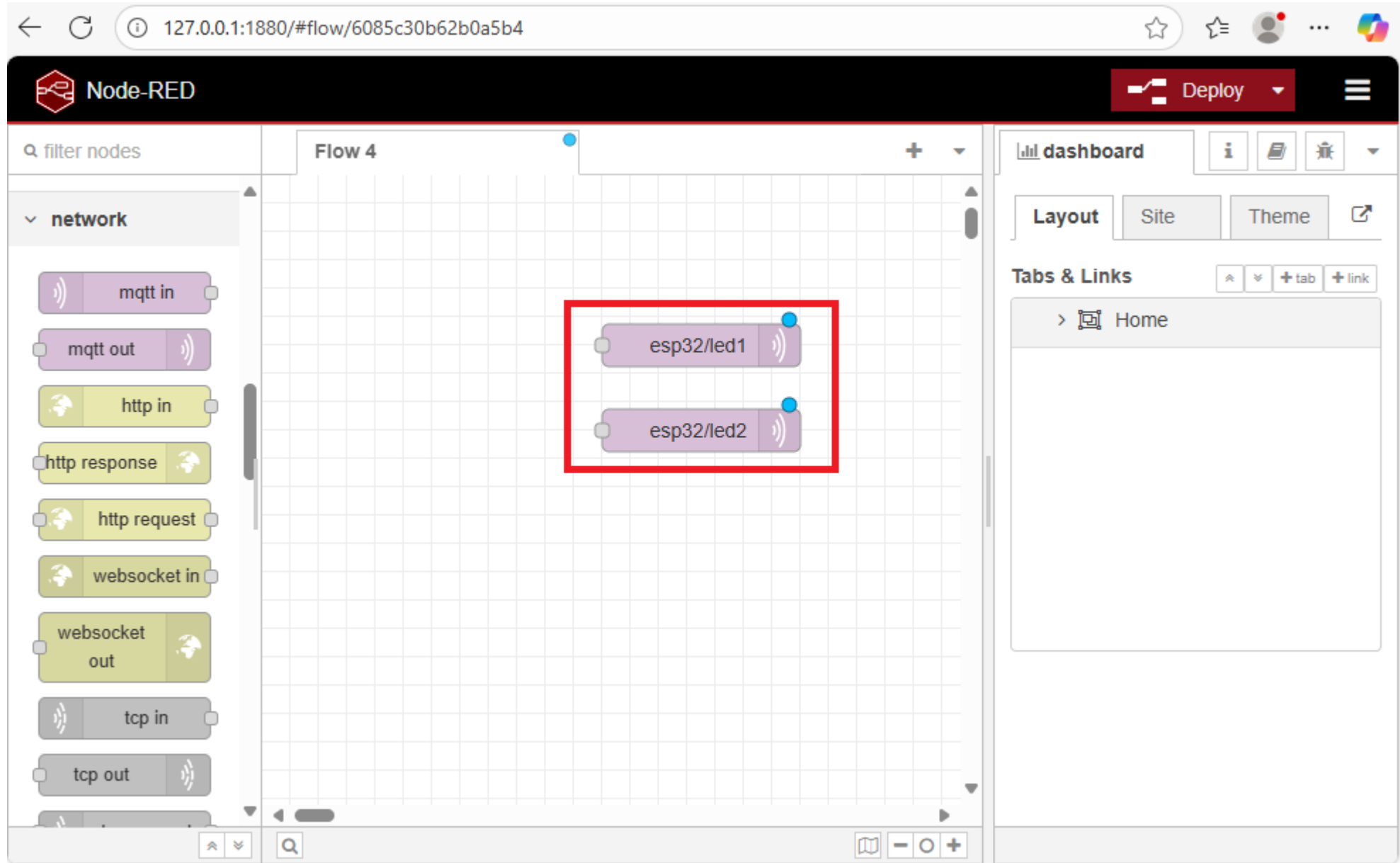


The screenshot shows the Node-RED web interface in a browser. The address bar displays the URL `127.0.0.1:1880/#flow/6085c30b62b0a5b4`. The main editor area is titled "Edit mqtt out node". In the top right of this editor, the "Done" button is highlighted with a red rectangle. Below the buttons, the "Properties" section is visible. The "Topic" field, which contains the text "esp32/led1", is also highlighted with a red rectangle. Other visible fields include "Server" (broker.hivemq.com:1883), "QoS" (0), and "Retain" (checked). A tip box at the bottom of the properties section states: "Tip: Leave topic, qos or retain blank if you want to set them via msg properties." The left sidebar shows various network-related nodes, and the right sidebar shows dashboard controls like "Layout", "Site", "Theme", and "Tabs & Links".

- The MQTT Out node is configured for **LED1**.

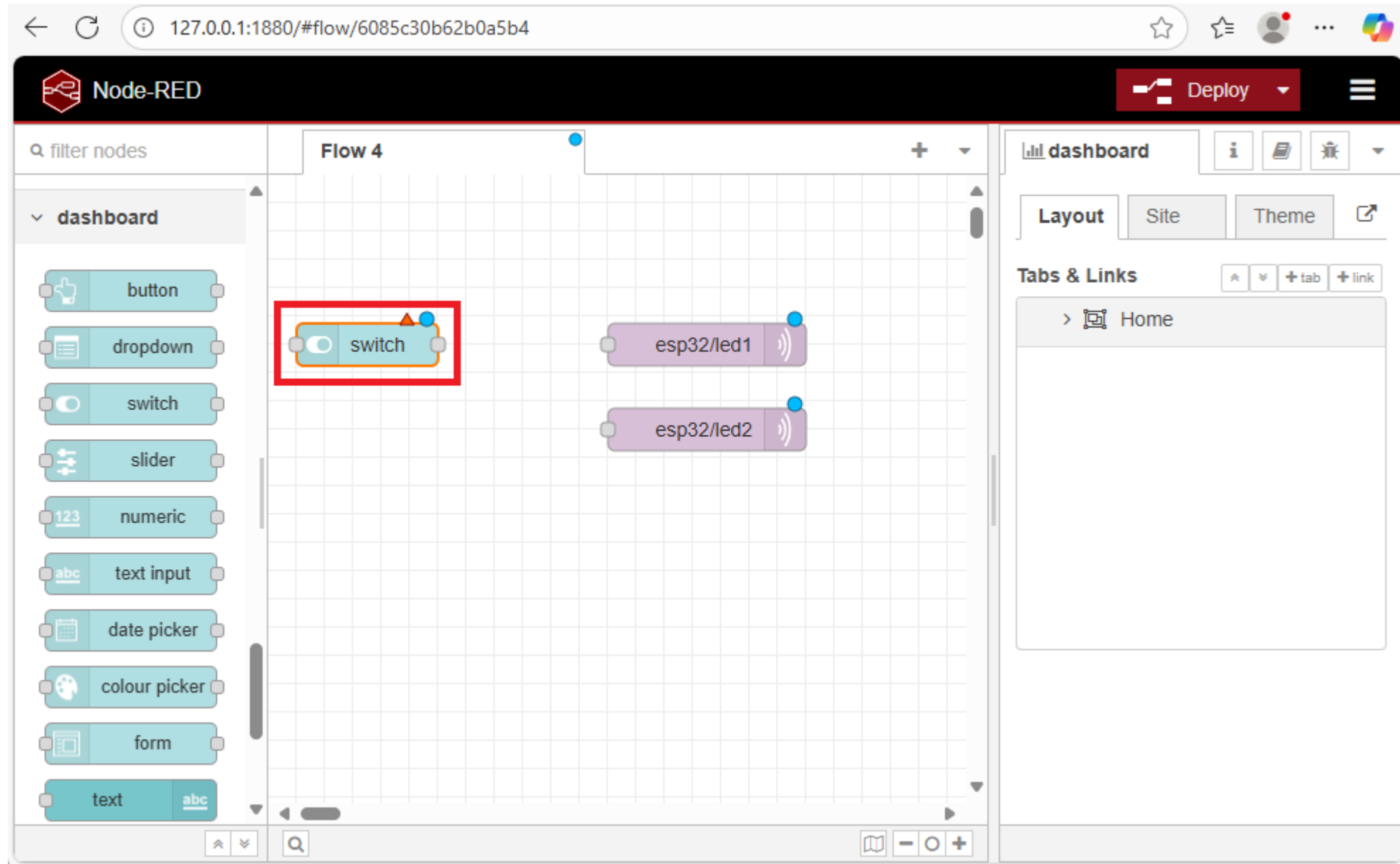


- Repeat the same procedure to configure the MQTT node for **Led2**.

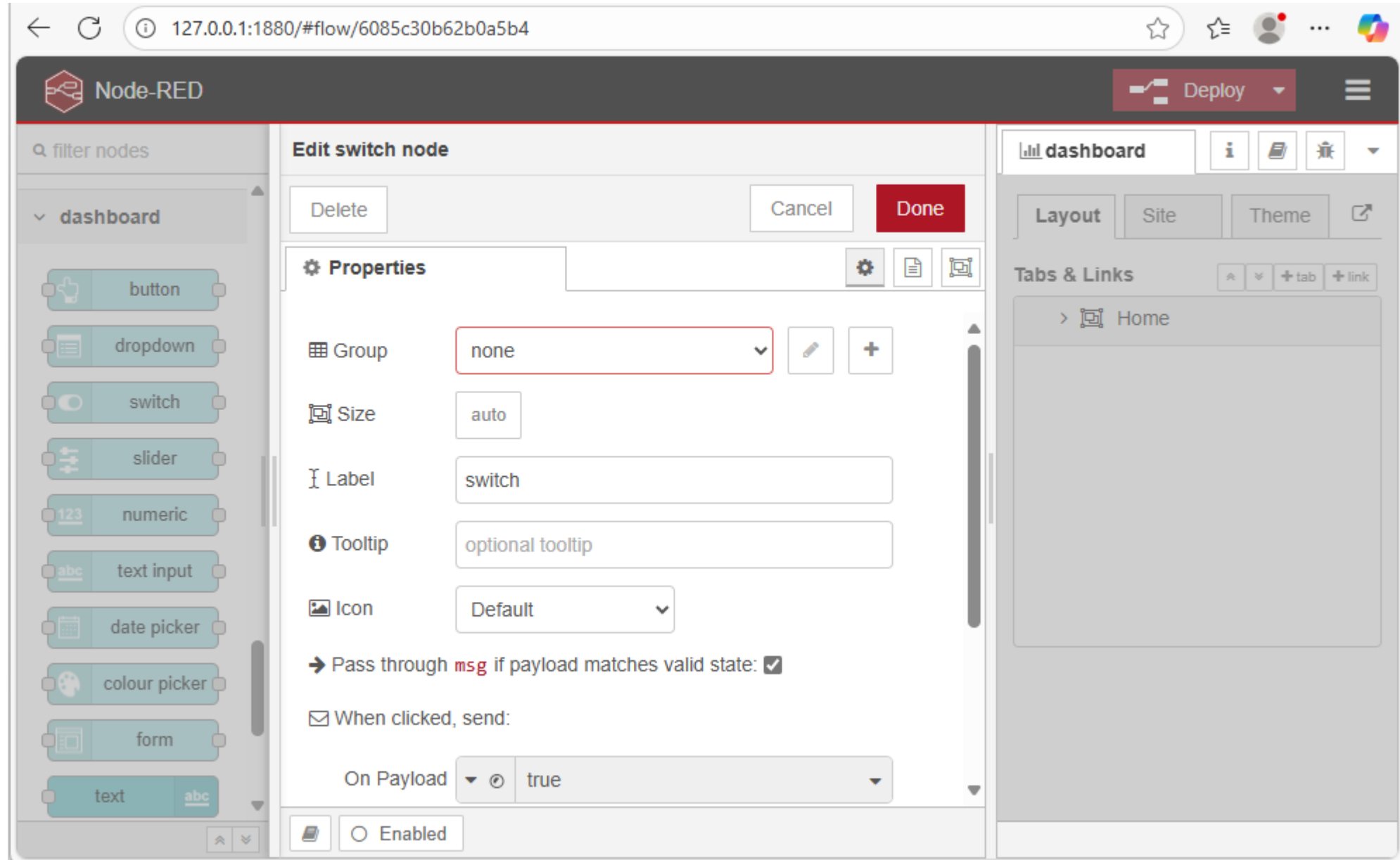


The screenshot shows the Node-RED web interface in a browser window. The address bar displays the URL `127.0.0.1:1880/#flow/6085c30b62b0a5b4`. The interface includes a top bar with the Node-RED logo, a "Deploy" button, and a hamburger menu. On the left, a sidebar lists various nodes under the "network" category, including "mqtt in", "mqtt out", "http in", "http response", "http request", "websocket in", "websocket out", "tcp in", and "tcp out". The main workspace, titled "Flow 4", contains two MQTT nodes: "esp32/led1" and "esp32/led2". These nodes are highlighted with a red rectangular box. The right sidebar shows a "dashboard" tab with options for "Layout", "Site", and "Theme", and a "Tabs & Links" section with a "Home" link.

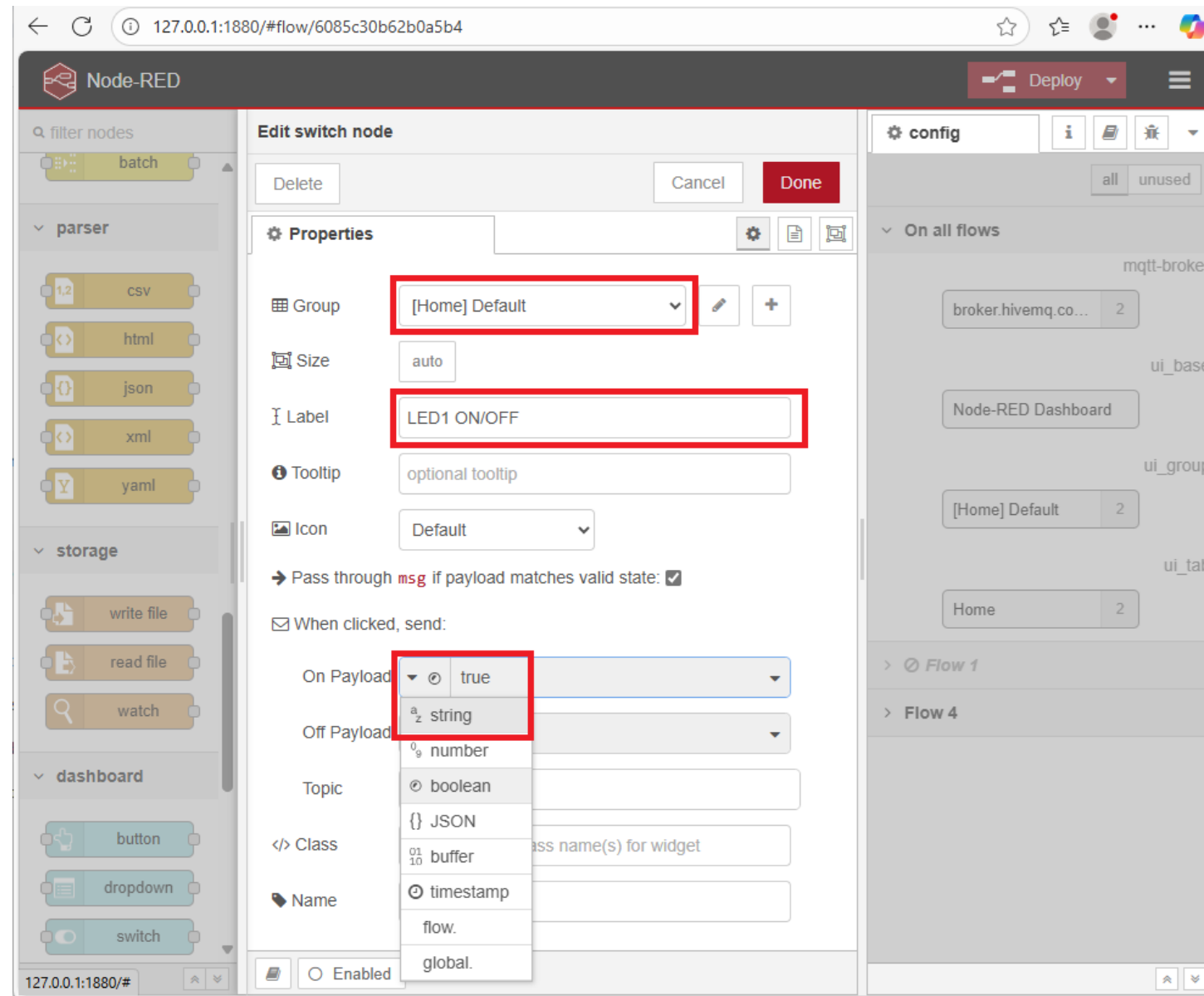
- Drag the **switch node** from the **dashboard** category into the **flow workspace**.



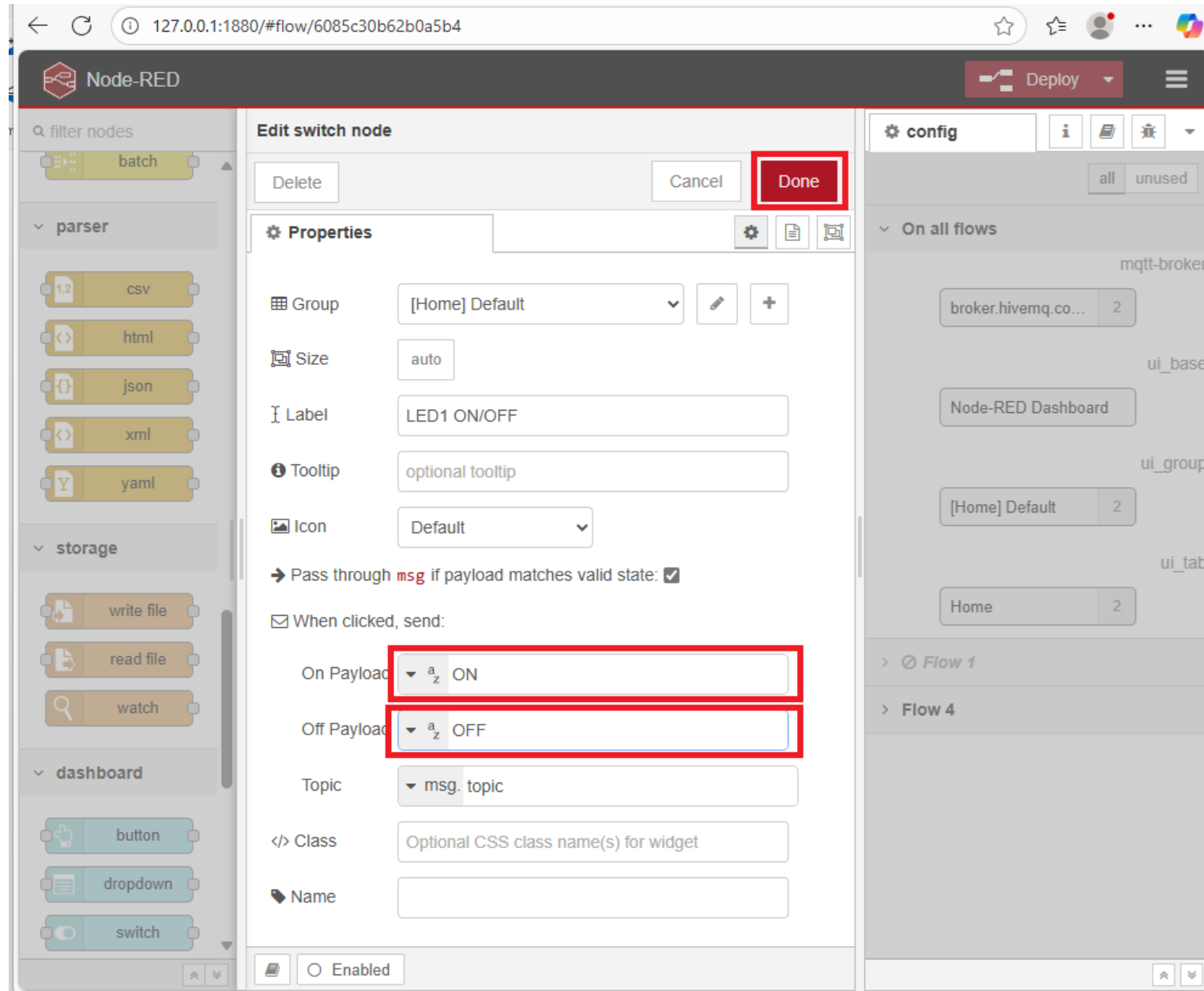
- Double-click the **switch node**; the **Edit Switch Node** window will open.



- Create a new group name or select the default group, change the label of the switch node, then click the down arrow next to **Payload** and select **String**.

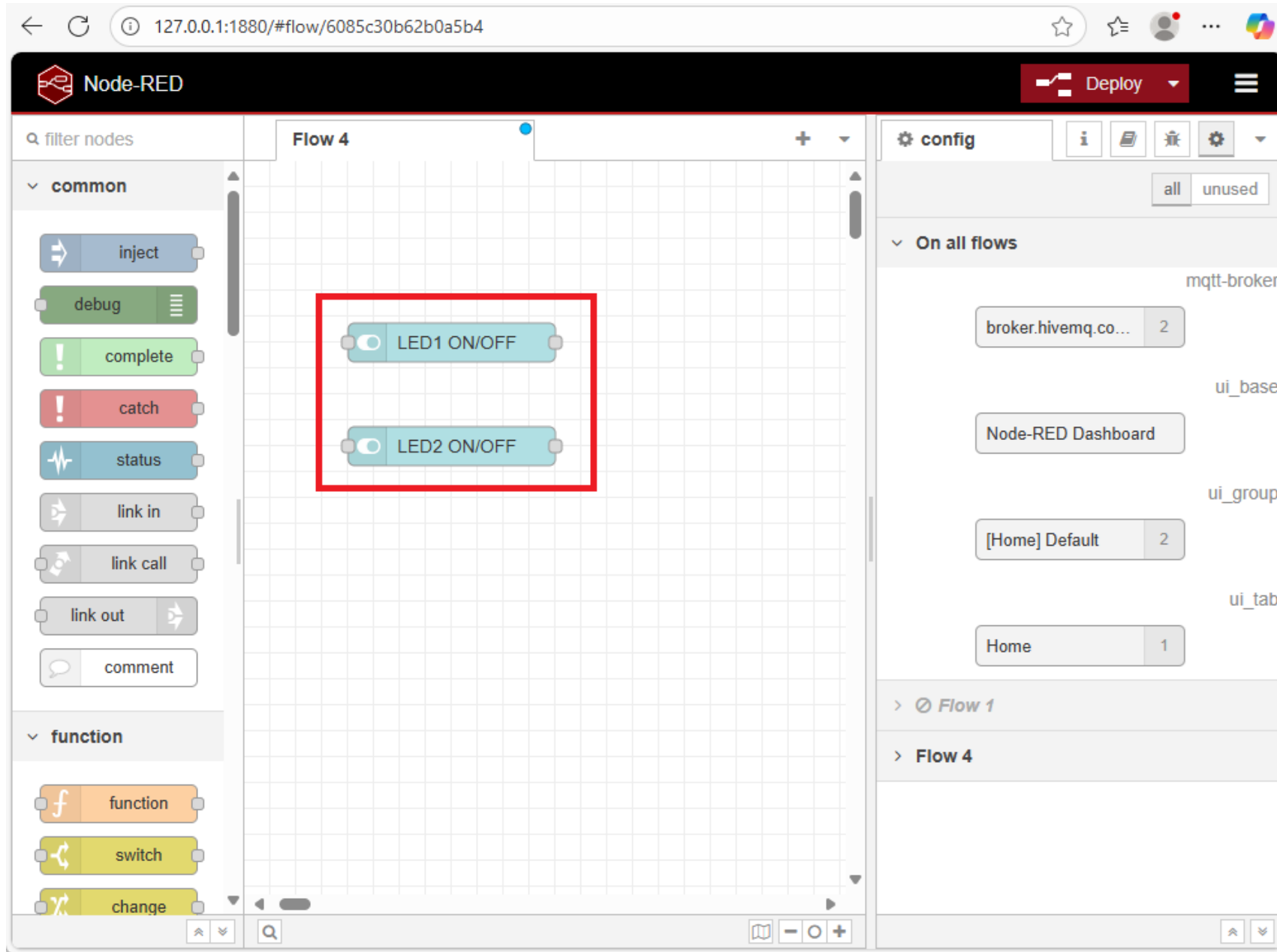


- Change the values of both payloads — one to **ON** and the other to **OFF** — then select the **Done** option.



The screenshot shows the Node-RED web interface. The 'Edit switch node' dialog is open, displaying various configuration options. The 'Done' button is highlighted with a red border. The 'On Payload' and 'Off Payload' fields are also highlighted with red borders, showing the values 'a_z ON' and 'a_z OFF' respectively. The 'Group' is set to '[Home] Default', 'Size' is 'auto', 'Label' is 'LED1 ON/OFF', and 'Icon' is 'Default'. The 'Pass through msg if payload matches valid state' checkbox is checked. The 'When clicked, send:' section is expanded, showing the 'On Payload' and 'Off Payload' fields. The 'Topic' is set to 'msg.topic'. The 'Class' is 'Optional CSS class name(s) for widget'. The 'Name' field is empty. The 'Enabled' checkbox is checked.

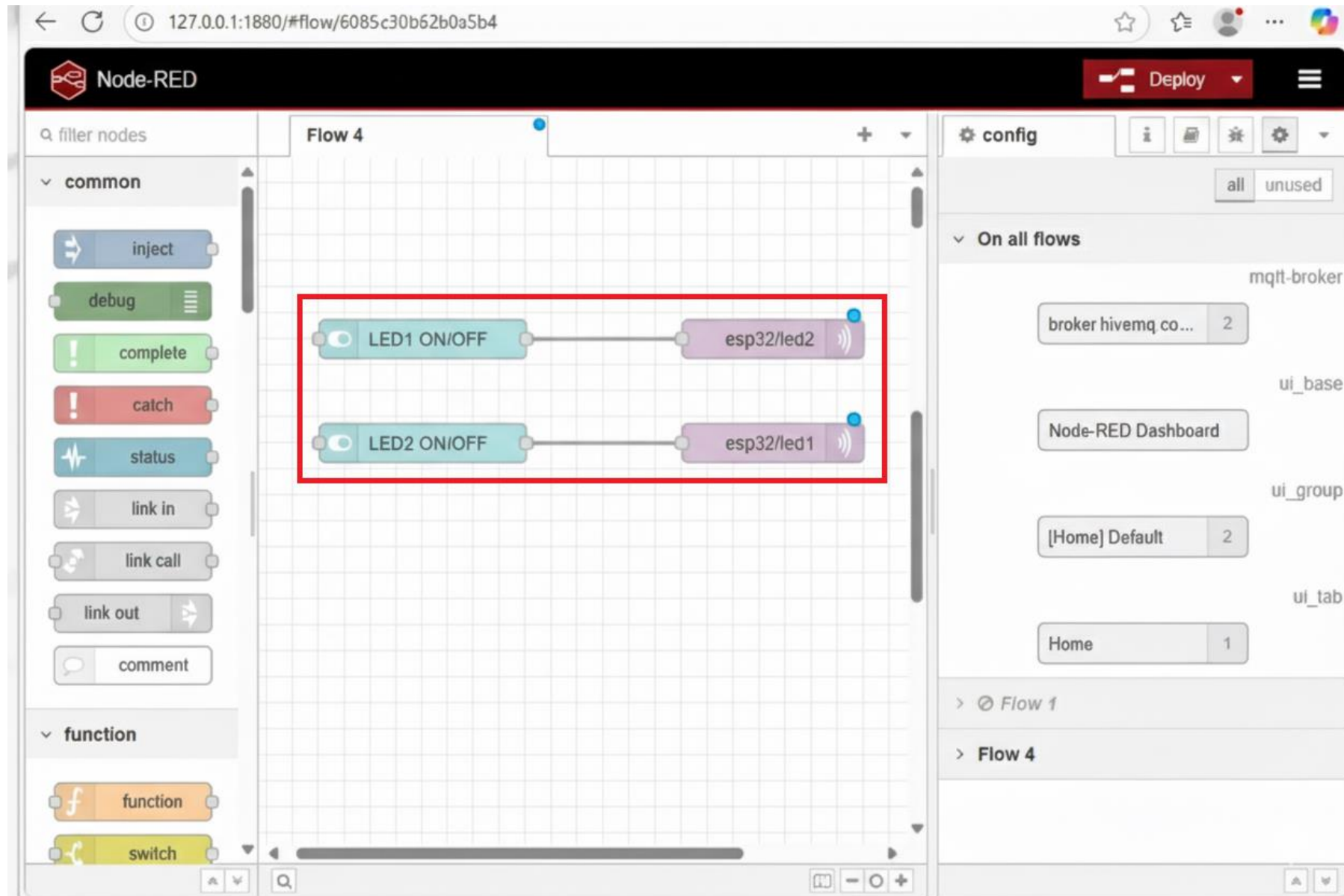
- Repeat the procedure to configure the other switch node.



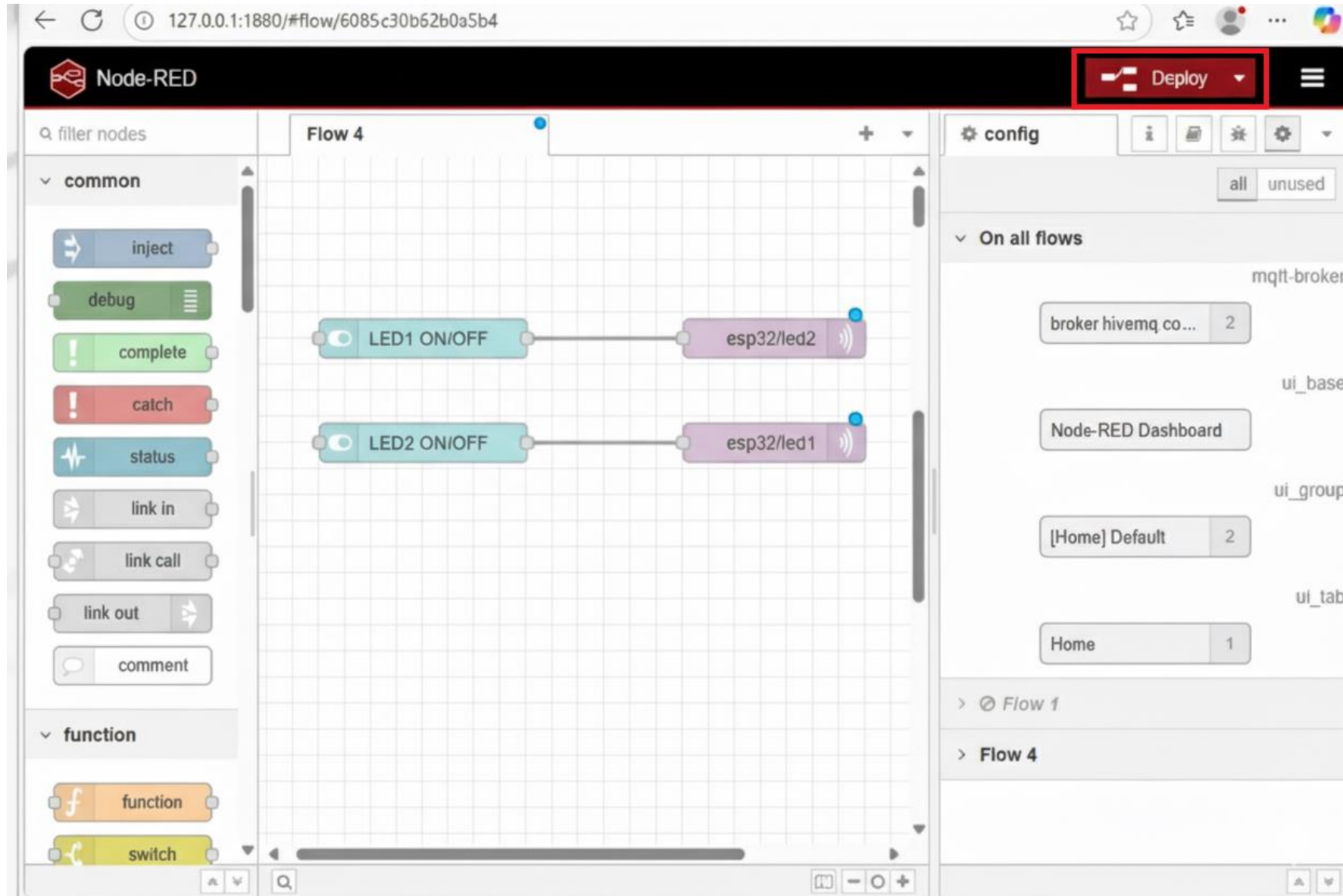
The screenshot shows the Node-RED web interface in a browser window. The address bar displays the URL `127.0.0.1:1880/#flow/6085c30b62b0a5b4`. The interface is divided into three main sections:

- Left Sidebar:** Contains a search bar and two categories of nodes:
 - common:** Includes nodes like `inject`, `debug`, `complete`, `catch`, `status`, `link in`, `link call`, `link out`, and `comment`.
 - function:** Includes nodes like `function`, `switch`, and `change`.
- Central Workspace:** Labeled "Flow 4", it contains a grid where two "LED ON/OFF" nodes are placed. The second node is highlighted with a red rectangle.
- Right Sidebar:** Labeled "config", it shows various settings:
 - mqtt-broker:** Set to `broker.hivemq.co...` with a value of `2`.
 - ui_base:** Set to `Node-RED Dashboard`.
 - ui_group:** Set to `[Home] Default` with a value of `2`.
 - ui_tab:** Set to `Home` with a value of `1`.

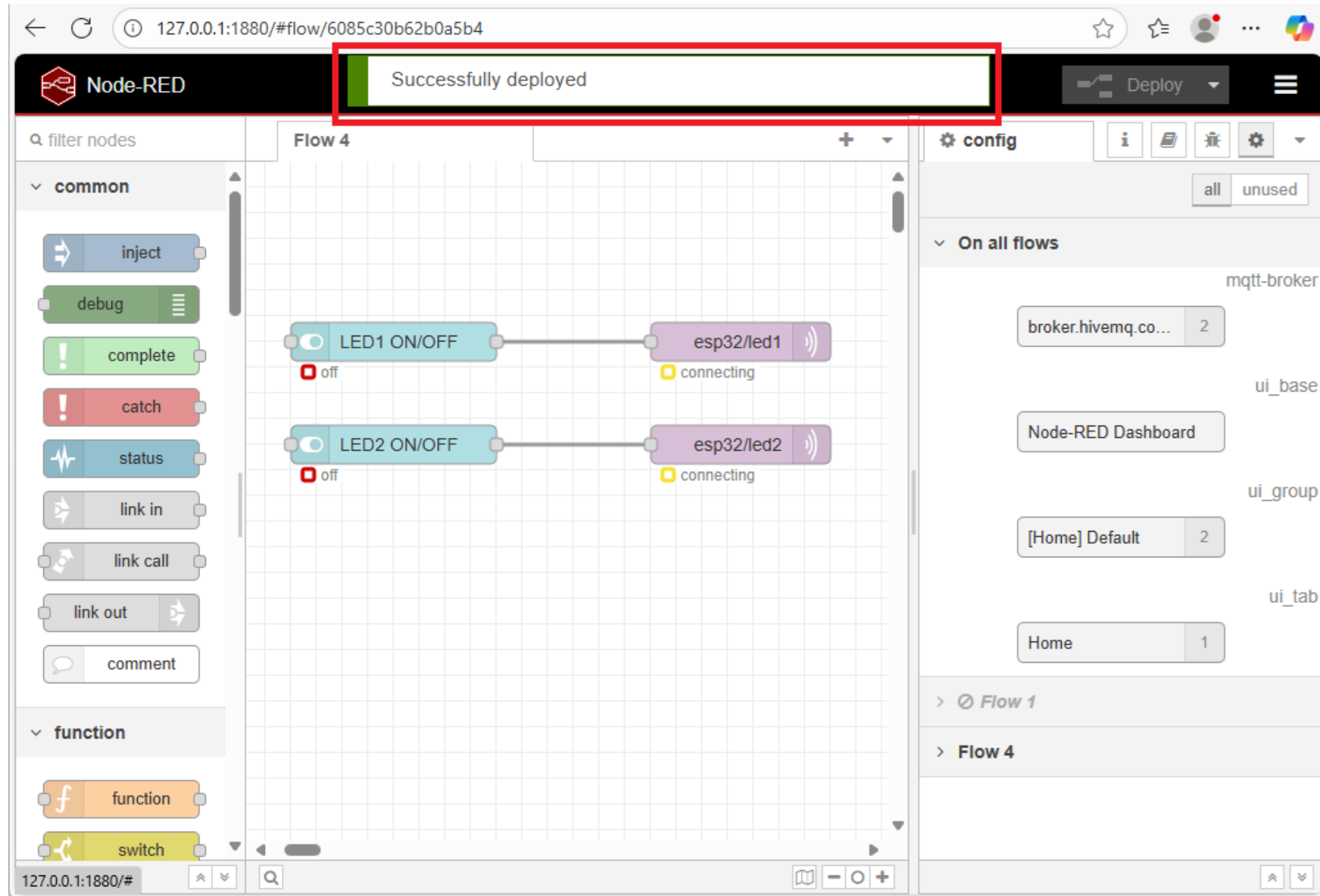
- Drag a **wire** from Button1 → MQTT out node for esp32/led1.
- Drag a **wire** from Button2 → MQTT out node for esp32/led2.



- **Click the Deploy Button:** Locate the red button labeled "Deploy" in the top-right corner of the Node-RED editor.



- **Confirmation:** A successful deployment will usually display a small notification at the top of the screen (e.g., "Successfully deployed").



127.0.0.1:1880/#flow/6085c30b62b0a5b4

Node-RED

Deploy

filter nodes

Flow 4

common

- inject
- debug
- complete
- catch
- status
- link in
- link call
- link out
- comment

function

LED1 ON/OFF

LED2 ON/OFF

esp32/led1

esp32/led2

config

all unused

On all flows

mqtt-broker

broker.hivemq.co... 2

ui_base

Node-RED Dashboard

ui_group

[Home] Default 2

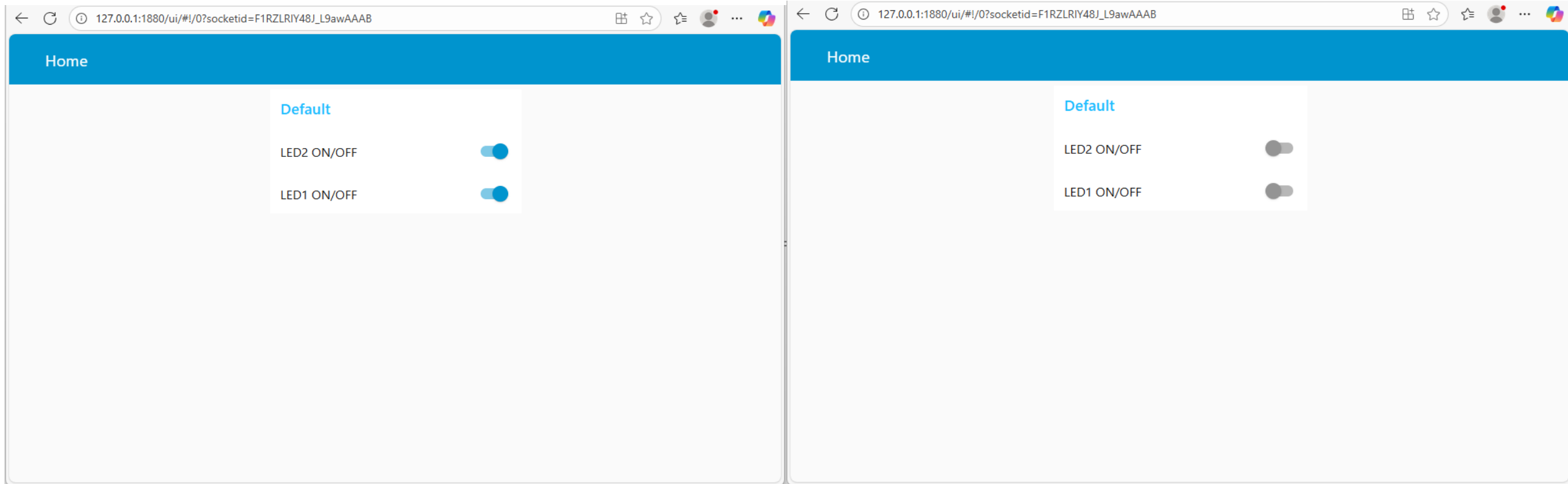
ui_tab

Home 2

Flow 1

Flow 4

- **Flow Activation:** Once deployed, the logic you wired—the Dashboard Switch publishing "ON" or "OFF" to the MQTT broker on topics esp32/led1 and esp32/led2—becomes active. Your ESP32, if connected to the same broker, will start responding to these commands.



MQTT Publishing Arduino Code

Potentiometer and LDR MQTT Code

```
#include <WiFi.h>
#include <PubSubClient.h>

// WiFi credentials
const char* ssid = "Pradeep";
const char* password = "123456789";

// MQTT broker info (HiveMQ public broker)
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* mqtt_topic = "iot/sensor";

// Analog pins for sensors
const int potPin = 36; // Example analog pin for potentiometer
const int ldrPin = 39; // Example analog pin for LDR

WiFiClient espClient;
PubSubClient client(espClient);
```

```
void setup_wifi()
{
    delay(10);
    Serial.println();
    Serial.print("Connecting to WiFi: ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi connected");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
}
```

```
void reconnect()
{
    // Loop until reconnected
    while (!client.connected())
    {
        Serial.print("Connecting to MQTT...");
        if (client.connect("ESP32Client"))
        {
            Serial.println("connected");
        }
        else
        {
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            delay(5000);
        }
    }
}
```

```
void setup()  
{  
    Serial.begin(115200);  
    setup_wifi();  
    client.setServer(mqtt_server, mqtt_port);  
}
```

```
void loop()  
{  
    if (!client.connected())  
    {  
        reconnect();  
    }  
    client.loop();  
  
    // Read analog values  
    int potValue = analogRead(potPin);  
    int ldrValue = analogRead(ldrPin);
```

```
// Construct JSON payload
```

```
String payload = "{";  
payload += "\"pot\":";  
payload += potValue;  
payload += ",";  
payload += "\"ldr\":";  
payload += ldrValue;  
payload += "}";
```

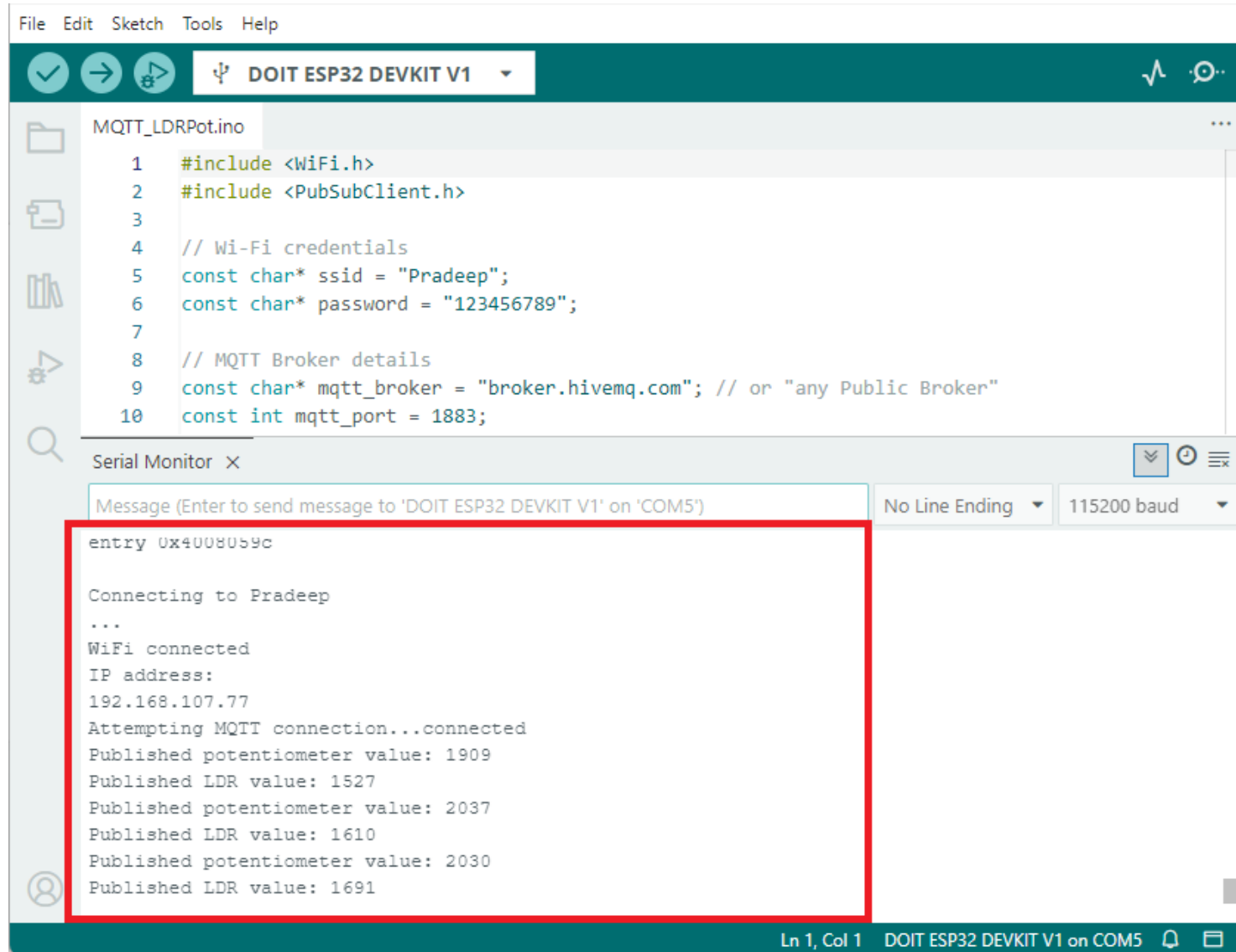
```
Serial.print("Publishing message: ");  
Serial.println(payload);
```

```
client.publish(mqtt_topic, payload.c_str());
```

```
delay(10000); // Wait 10 seconds (should match Telegraf interval)
```

```
}
```

- After uploading the code to the ESP32, once the upload is completed, press the reset button. The ESP32 will connect to Wi-Fi, print the IP address, and establish a connection with the MQTT broker.



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for checking, running, and uploading code. The selected board is DOIT ESP32 DEVKIT V1. The code editor displays the MQTT_LDRPot.ino file with the following code:

```

1  #include <WiFi.h>
2  #include <PubSubClient.h>
3
4  // Wi-Fi credentials
5  const char* ssid = "Pradeep";
6  const char* password = "123456789";
7
8  // MQTT Broker details
9  const char* mqtt_broker = "broker.hivemq.com"; // or "any Public Broker"
10 const int mqtt_port = 1883;

```

The Serial Monitor window is open, showing the output of the program. The output is as follows:

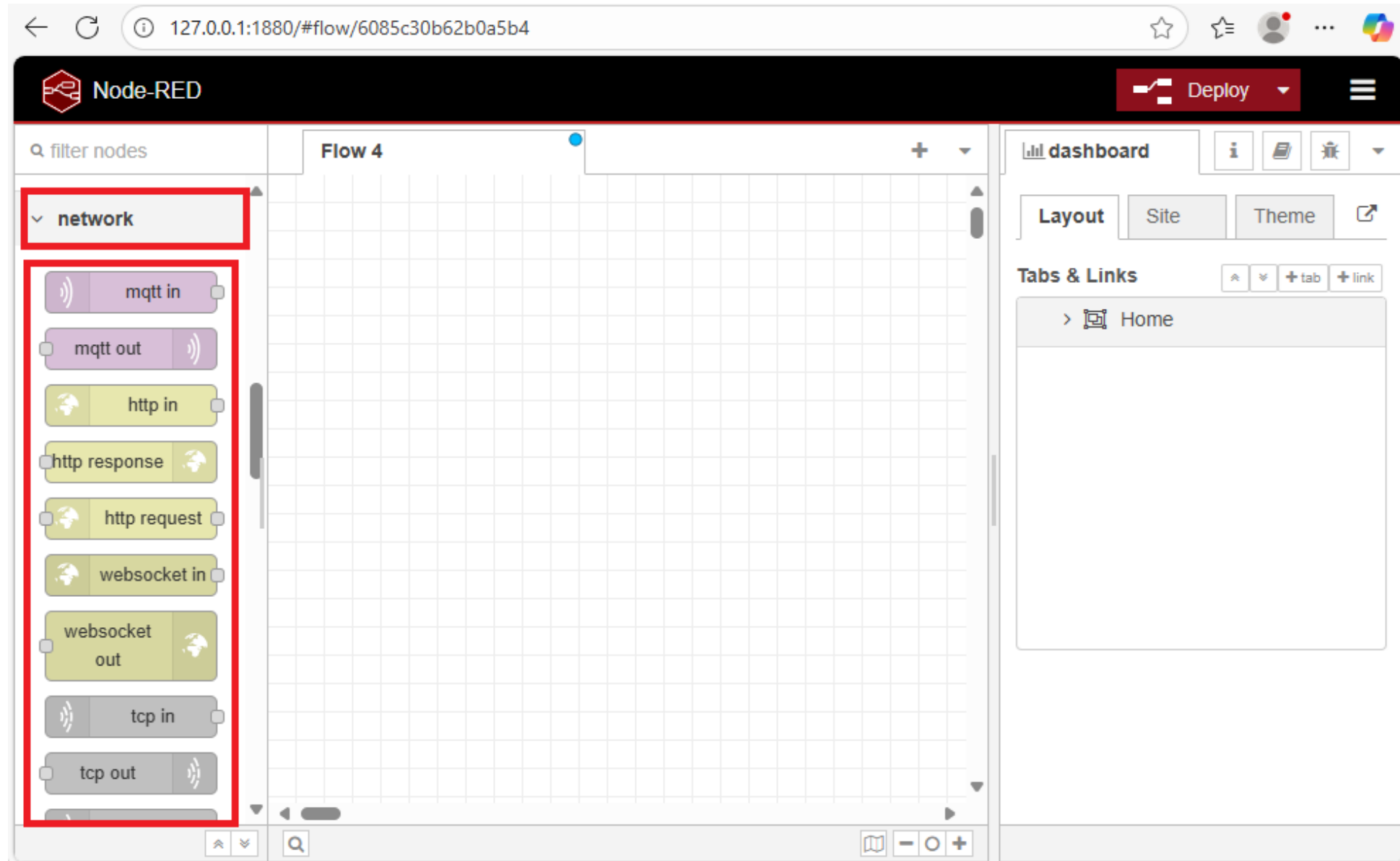
```

entry 0x4008059c
Connecting to Pradeep
...
WiFi connected
IP address:
192.168.107.77
Attempting MQTT connection...connected
Published potentiometer value: 1909
Published LDR value: 1527
Published potentiometer value: 2037
Published LDR value: 1610
Published potentiometer value: 2030
Published LDR value: 1691

```

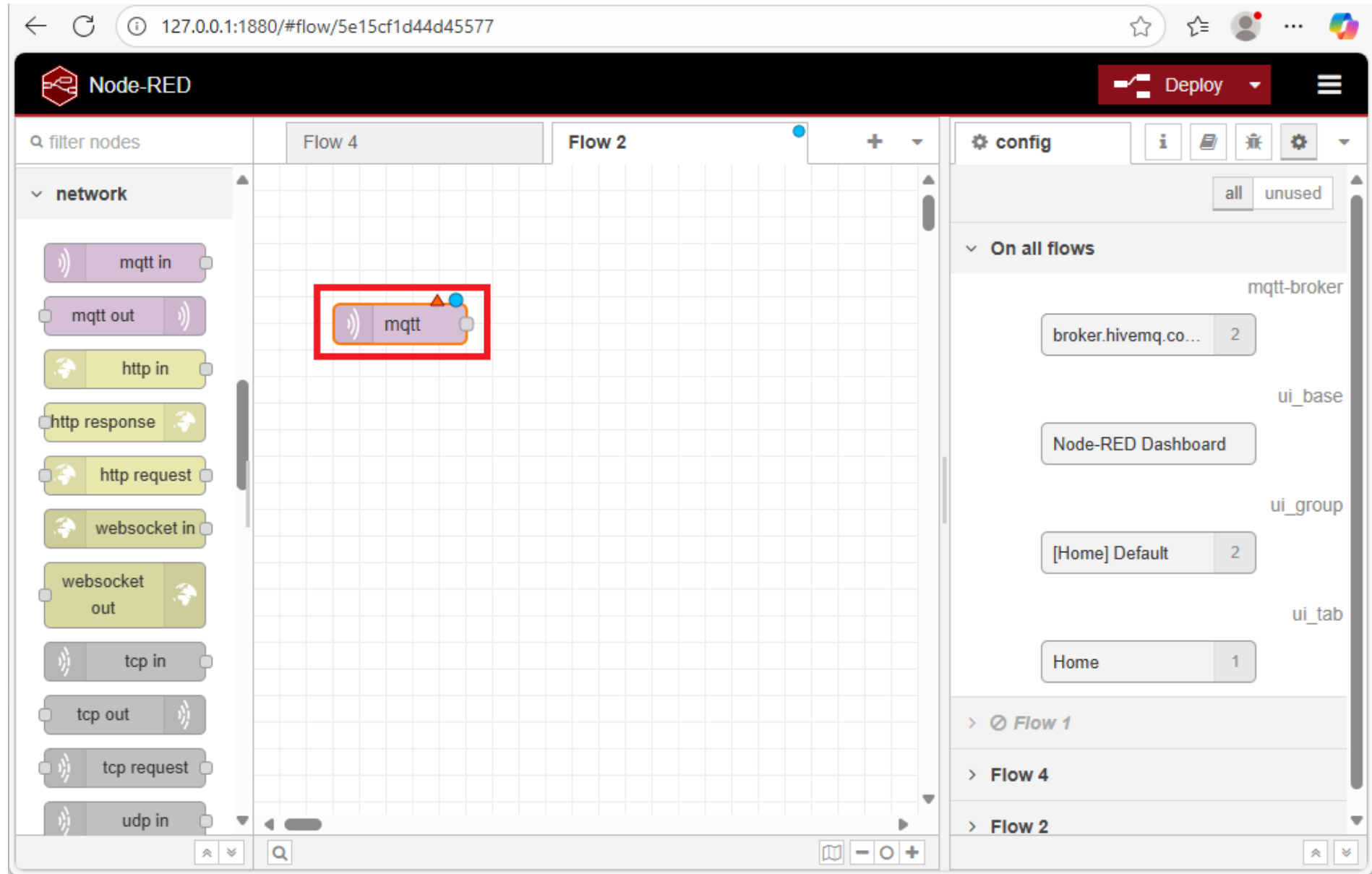
The Serial Monitor window has a message input field and a dropdown menu for line ending (set to No Line Ending) and a baud rate dropdown (set to 115200 baud). The status bar at the bottom indicates the current position is Ln 1, Col 1 on the DOIT ESP32 DEVKIT V1 on COM5.

- In the left palette, scroll down until you find the "network" category, then drag the **MQTT in** node to the workflow space.



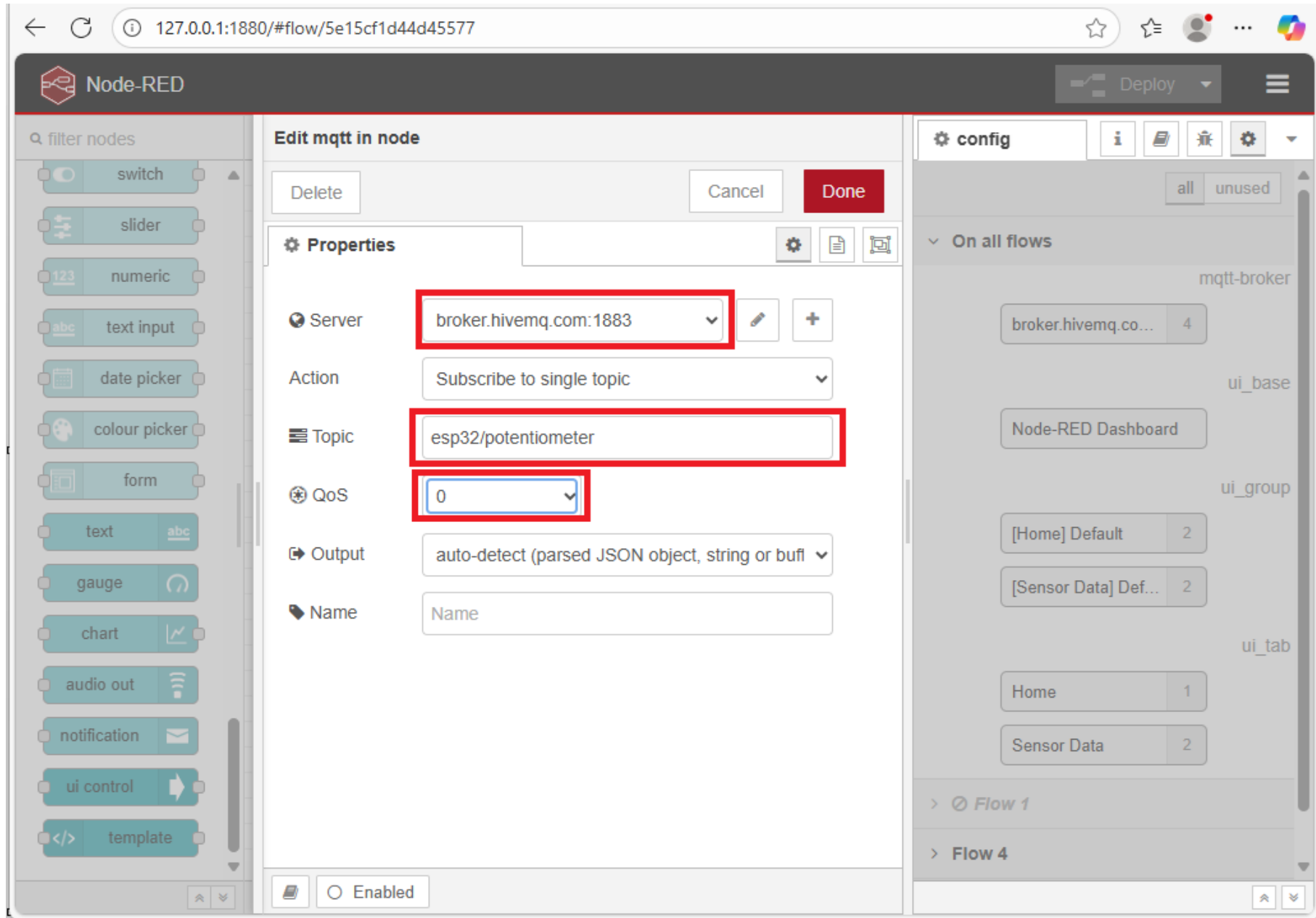
Configure MQTT connection

- Double-click the MQTT node to open its configuration settings.



The screenshot shows the Node-RED web interface in a browser window. The address bar displays '127.0.0.1:1880/#flow/5e15cf1d44d45577'. The interface includes a left sidebar with a 'filter nodes' search bar and a 'network' category expanded, showing various nodes like 'mqtt in', 'mqtt out', 'http in', 'http response', 'http request', 'websocket in', 'websocket out', 'tcp in', 'tcp out', 'tcp request', and 'udp in'. The central workspace, labeled 'Flow 2', contains a single 'mqtt' node highlighted with a red rectangular box. The right sidebar shows the 'config' tab for the selected node, with a list of configuration items under 'On all flows': 'mqtt-broker' (broker.hivemq.co... 2), 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default 2), and 'ui_tab' (Home 1). At the bottom of the right sidebar, a list of flows is visible: 'Flow 1', 'Flow 4', and 'Flow 2'.

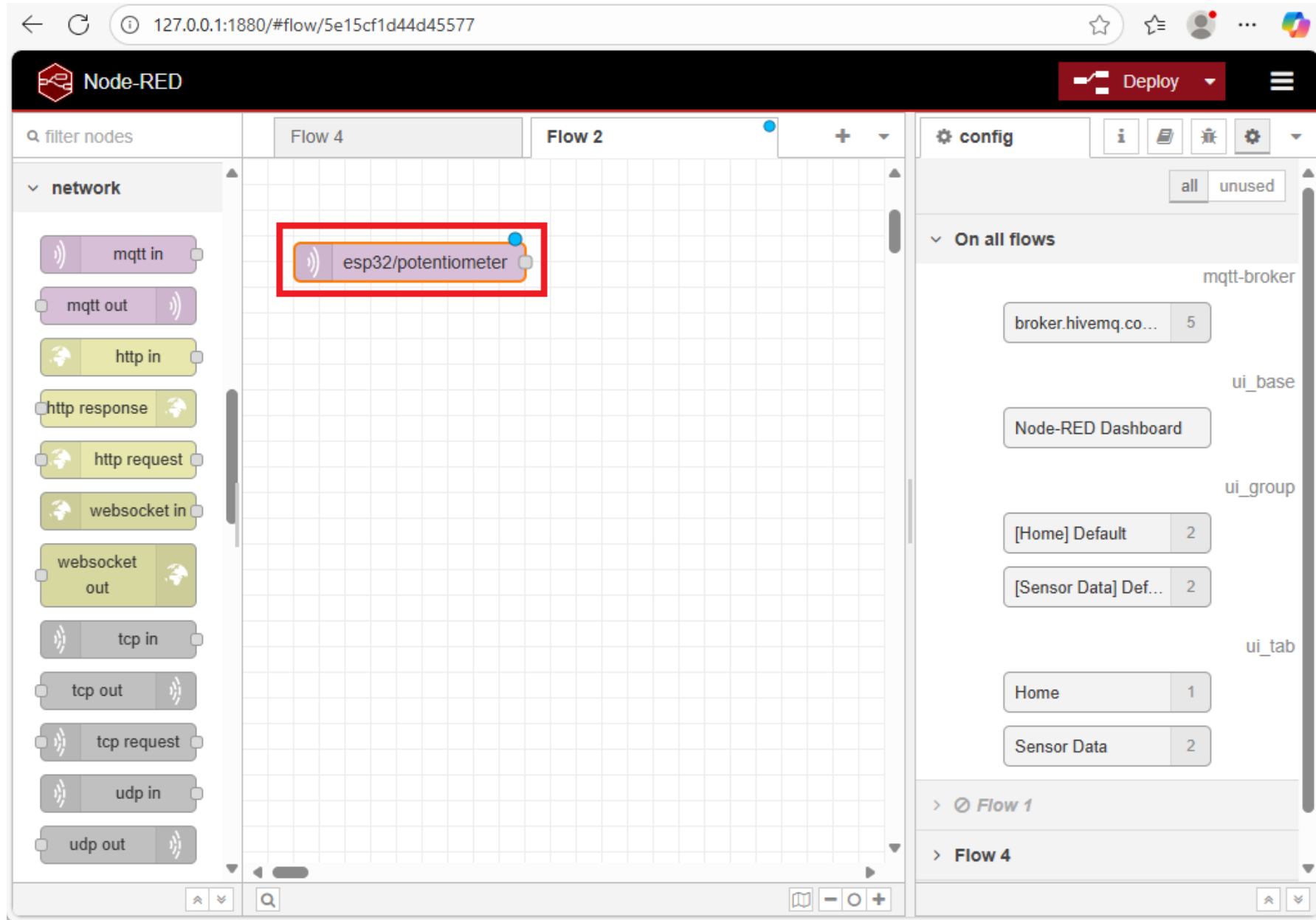
- Add the MQTT server, port, and topic, then select the "Done" option. **(Potentiometer)**



The screenshot shows the Node-RED web interface in a browser. The address bar displays the URL: 127.0.0.1:1880/#flow/5e15cf1d44d45577. The interface is divided into three main sections:

- Left Panel (Nodes):** A list of available nodes including switch, slider, numeric, text input, date picker, colour picker, form, text, gauge, chart, audio out, notification, ui control, and template.
- Center Panel (Edit mqtt in node):** This panel is used to configure the MQTT node. It includes a "Delete" button, "Cancel" and "Done" buttons, and a "Properties" section. The "Properties" section contains the following fields:
 - Server:** A dropdown menu showing "broker.hivemq.com:1883", which is highlighted with a red box.
 - Action:** A dropdown menu showing "Subscribe to single topic".
 - Topic:** A text input field containing "esp32/potentiometer", which is highlighted with a red box.
 - QoS:** A dropdown menu showing "0", which is highlighted with a red box.
 - Output:** A dropdown menu showing "auto-detect (parsed JSON object, string or bufl)".
 - Name:** A text input field containing "Name".
- Right Panel (config):** This panel shows the configuration for the current flow. It includes a "Deploy" button and a "config" tab. The "config" tab shows a list of nodes and their configurations, including "mqtt-broker", "ui_base", "ui_group", and "ui_tab".

- The MQTT input node has been configured in Node-RED (Potentiometer)



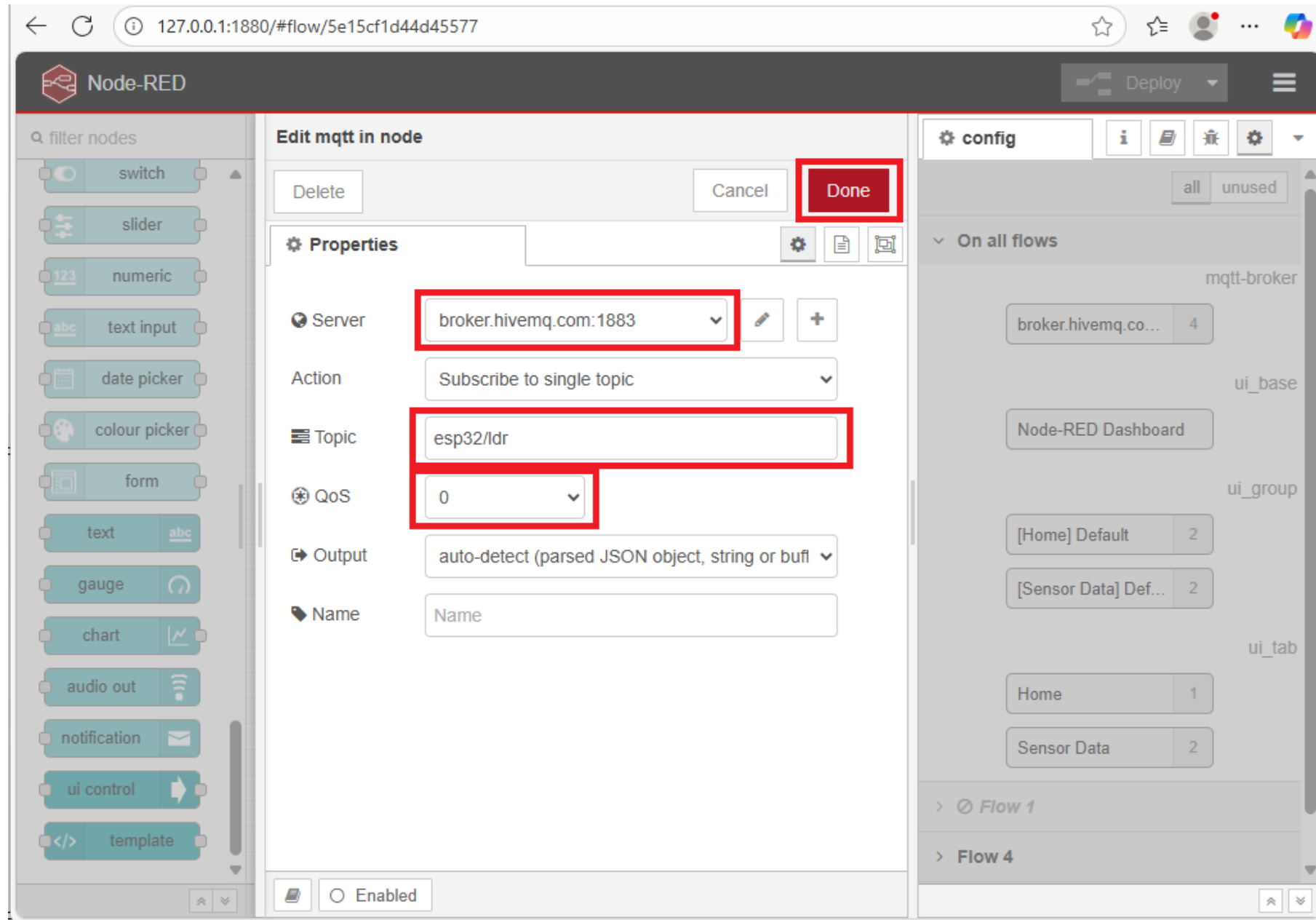
The screenshot displays the Node-RED web interface in a browser window. The address bar shows the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface includes a left sidebar with a 'filter nodes' search bar and a 'network' category expanded, listing various input and output nodes. The central workspace, titled 'Flow 2', contains a single node labeled 'esp32/potentiometer', which is highlighted with a red rectangular box. The right sidebar shows the 'config' tab for the selected node, displaying settings for 'mqtt-broker' (broker.hivemq.co...), 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default), and 'ui_tab' (Home). The bottom of the interface features a status bar with zoom controls and a 'Deploy' button.

- Add another MQTT input node to the flow workspace

The screenshot displays the Node-RED web interface in a browser window. The address bar shows the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface is divided into three main sections:

- Left Panel (Nodes):** A sidebar with a search bar labeled "filter nodes". Under the "network" category, various nodes are listed. The "mqtt" node is highlighted with a red rectangular box, indicating it is the node being added to the flow.
- Center Panel (Workspace):** A grid-based workspace where two nodes are already present: "esp32/potentiometer" and "mqtt". The "mqtt" node is the one being added from the left panel.
- Right Panel (Configuration):** A sidebar for configuring the flow. It includes a "config" tab, a "Deploy" button, and a list of nodes used in the flow. The "mqtt-broker" section is expanded, showing the configuration for the MQTT broker (broker.hivemq.co...). Other sections like "ui_base", "ui_group", and "ui_tab" are also visible.

- Add the MQTT server, port, and topic, then select the "Done" option. **(LDR)**



- The MQTT input node has been configured in Node-RED (LDR)

The screenshot displays the Node-RED web interface in a browser window. The address bar shows the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface is divided into several sections:

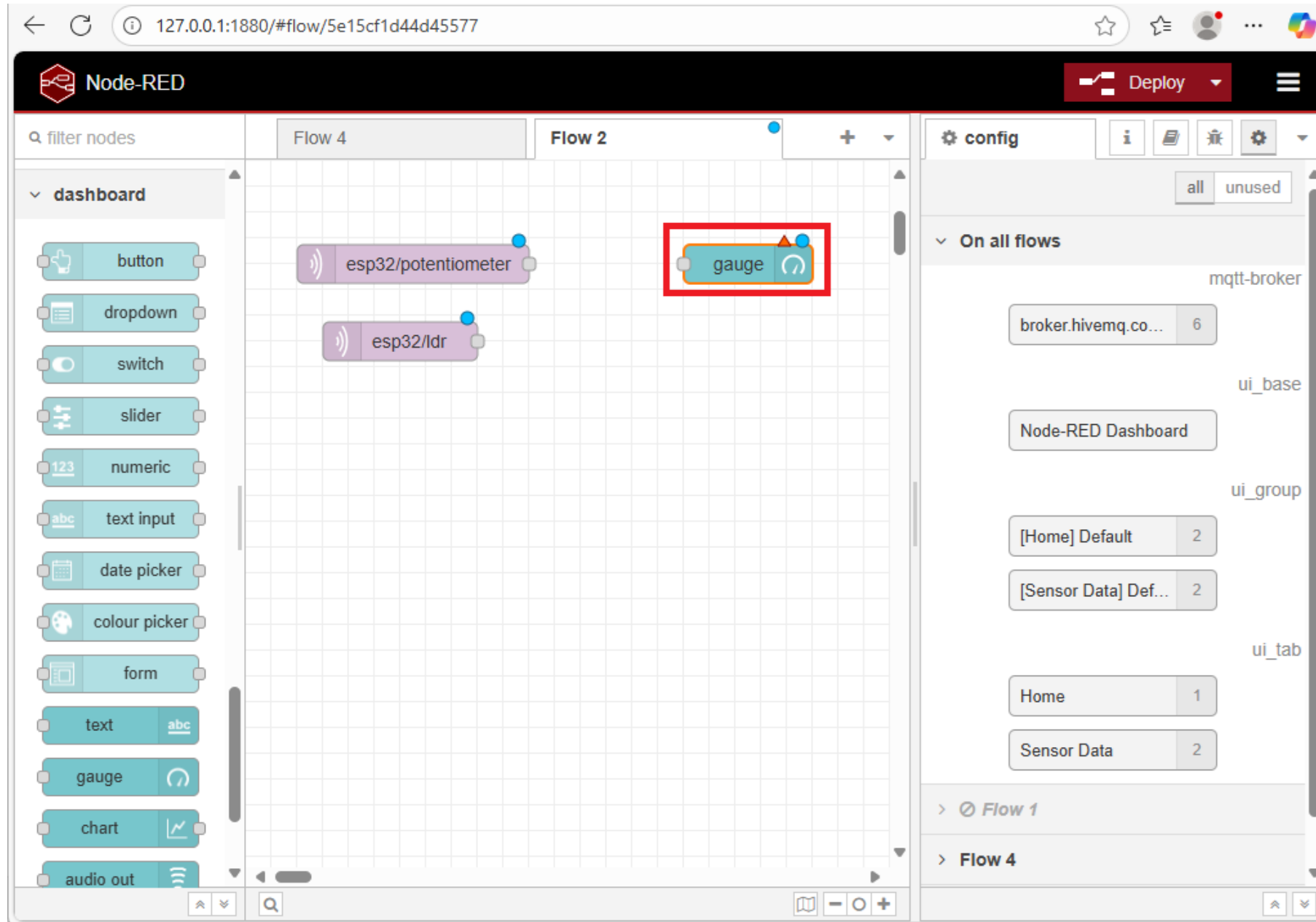
- Left Panel (Nodes):** A list of nodes categorized under 'network'. The 'mqtt in' node is highlighted, indicating it is the selected node for configuration.
- Center Canvas:** A workspace for building flows. It contains two nodes: 'esp32/potentiometer' and 'esp32/ldr'. The 'esp32/ldr' node is highlighted with a red rectangle, indicating it is the selected node for configuration.
- Right Panel (Configuration):** A configuration panel for the selected 'esp32/ldr' node. It shows various settings for the MQTT input node, including the broker address, port, and topic.

The configuration panel on the right shows the following settings:

- mqtt-broker:**
 - broker.hivemq.co... 6
- ui_base:**
 - Node-RED Dashboard
- ui_group:**
 - [Home] Default 2
 - [Sensor Data] Def... 2
- ui_tab:**
 - Home 1
 - Sensor Data 2

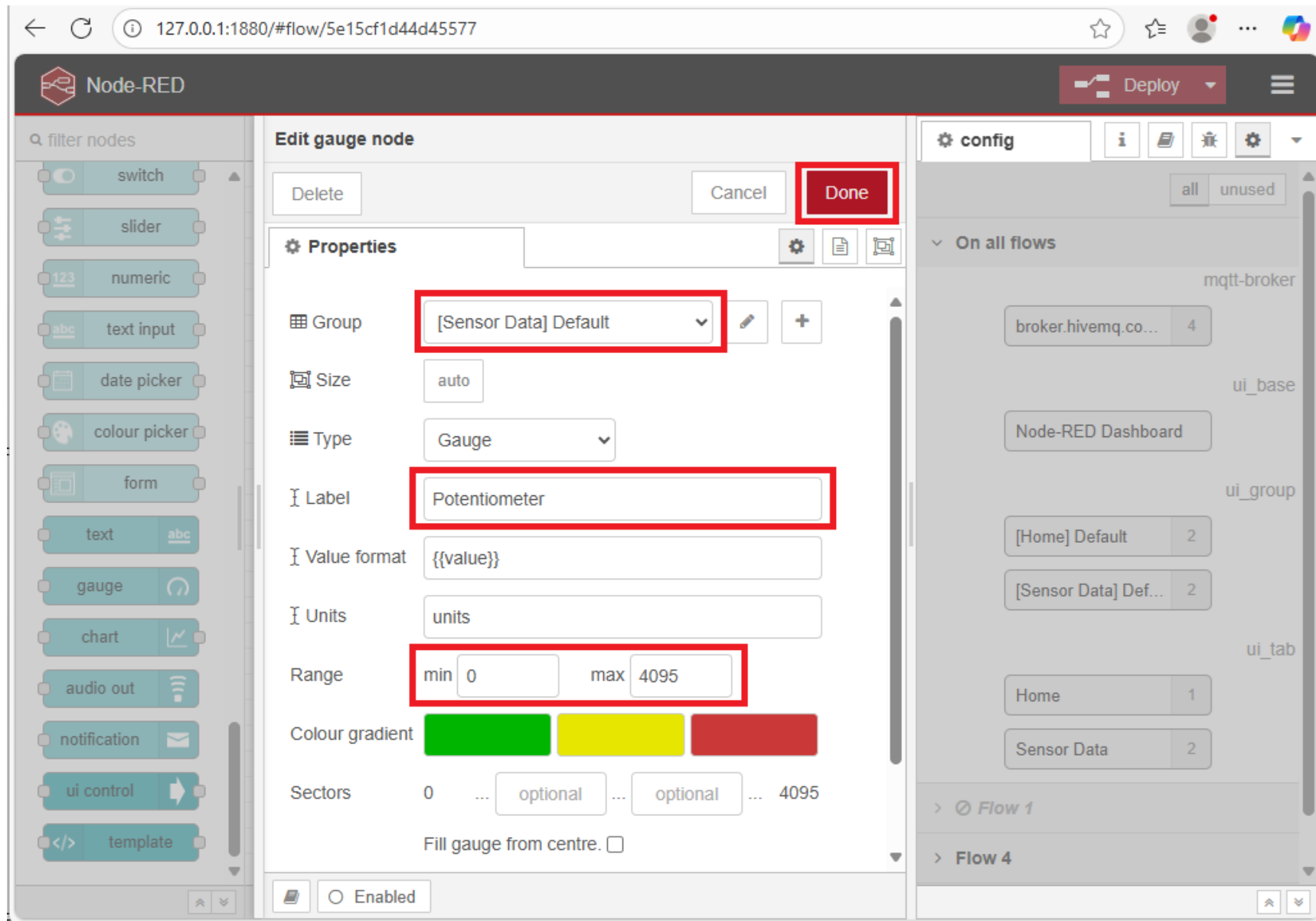
The bottom of the interface shows a list of flows: 'Flow 1' and 'Flow 4'.

- Drag the **Gauge** node from the **dashboard** category into the **flow workspace**.

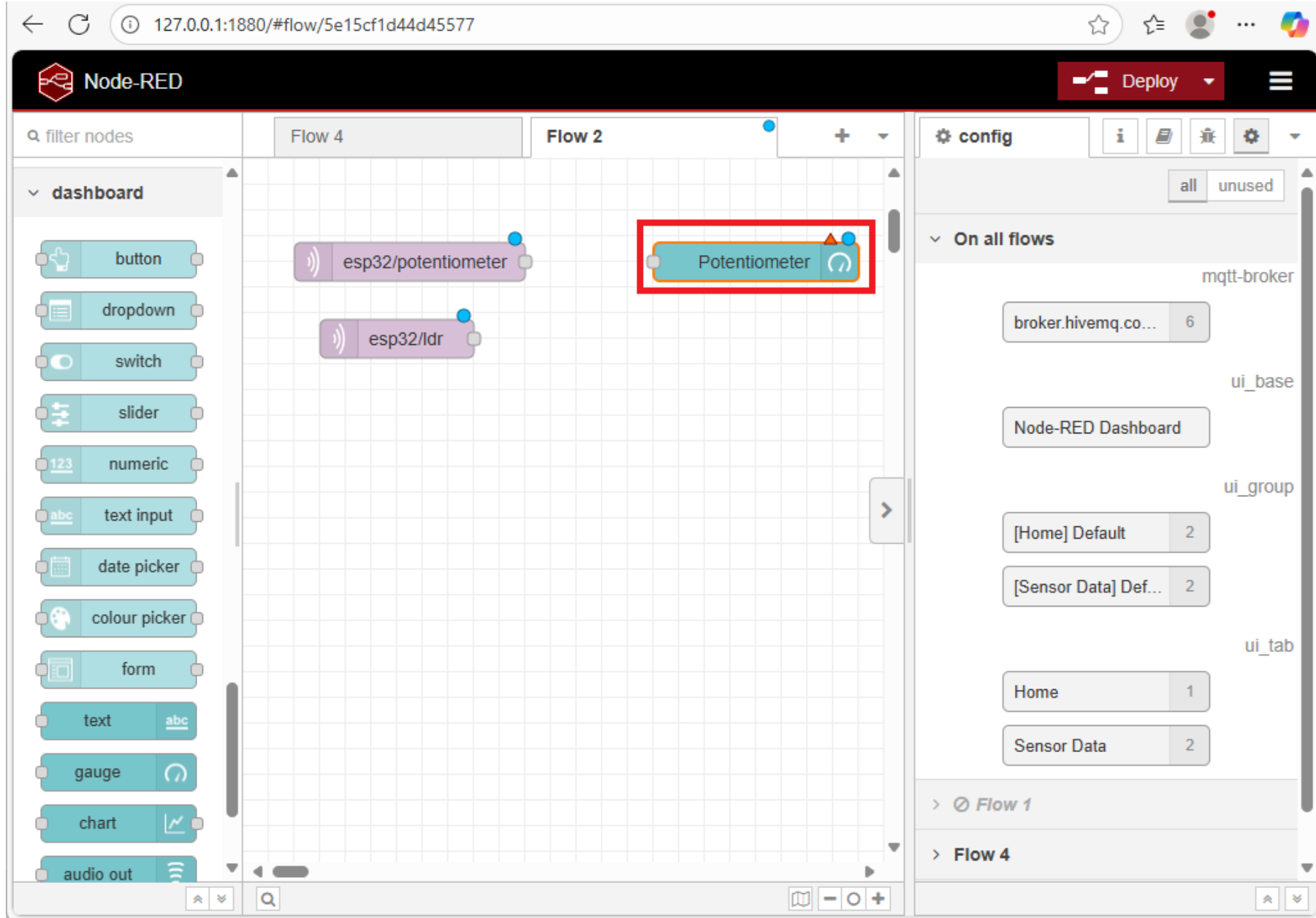


The screenshot shows the Node-RED web interface in a browser window. The address bar displays the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface includes a top navigation bar with the Node-RED logo, a 'Deploy' button, and a hamburger menu. The left sidebar contains a 'filter nodes' search bar and a 'dashboard' category expanded to show various dashboard nodes: button, dropdown, switch, slider, numeric, text input, date picker, colour picker, form, text, gauge, chart, and audio out. The central flow workspace, currently showing 'Flow 2', contains two 'esp32' nodes: 'esp32/potentiometer' and 'esp32/ldr'. A 'gauge' node is being dragged from the 'dashboard' category into the workspace, highlighted by a red rectangular box. The right sidebar shows the 'config' tab with settings for 'mqtt-broker', 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default and [Sensor Data] Def...), and 'ui_tab' (Home and Sensor Data). The bottom of the interface features a status bar with zoom controls and a search icon.

- Add the group name, label, and range values in the gauge node's edit window, then select **Done (Potentiometer)**



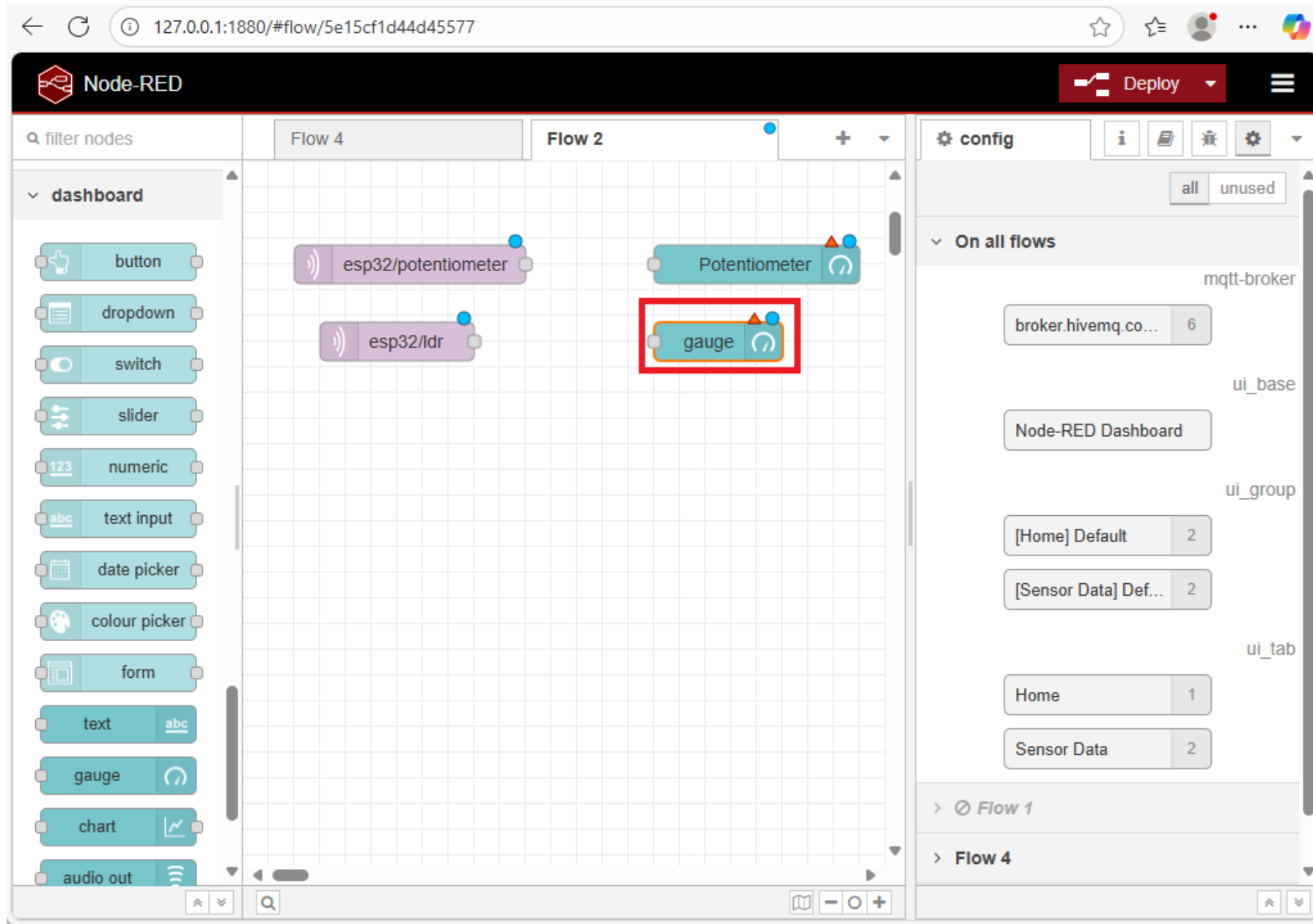
- Potentiometer gauge is configured



The screenshot displays the Node-RED web interface in a browser window. The address bar shows the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface is divided into three main sections:

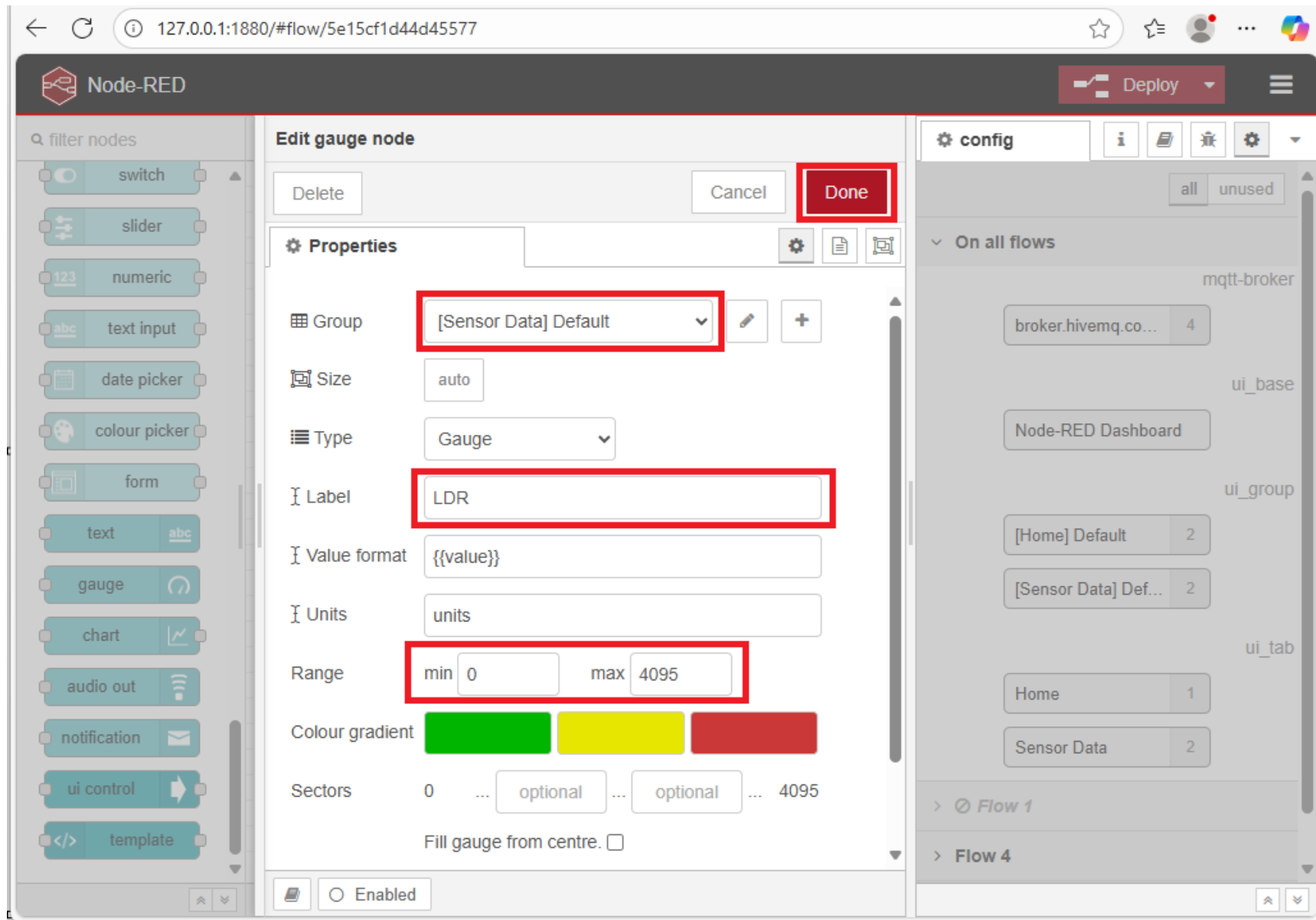
- Left Panel (Nodes):** A sidebar with a search bar and a list of nodes under the 'dashboard' category. The nodes include: button, dropdown, switch, slider, numeric, text input, date picker, colour picker, form, text, gauge, chart, and audio out. The 'gauge' node is highlighted with a red box.
- Center Panel (Canvas):** A workspace with a grid background. It contains two flow tabs: 'Flow 4' and 'Flow 2'. In 'Flow 2', there are two nodes: 'esp32/potentiometer' and 'esp32/ldr'. The 'Potentiometer' node is highlighted with a red box.
- Right Panel (Configuration):** A sidebar with a 'config' tab. It shows settings for 'On all flows' and 'Flow 4'. The 'On all flows' section includes:
 - mqtt-broker:** broker.hivemq.co... 6
 - ui_base:** Node-RED Dashboard
 - ui_group:** [Home] Default 2, [Sensor Data] Def... 2
 - ui_tab:** Home 1, Sensor Data 2

- Add another gauge node to the flow workspace

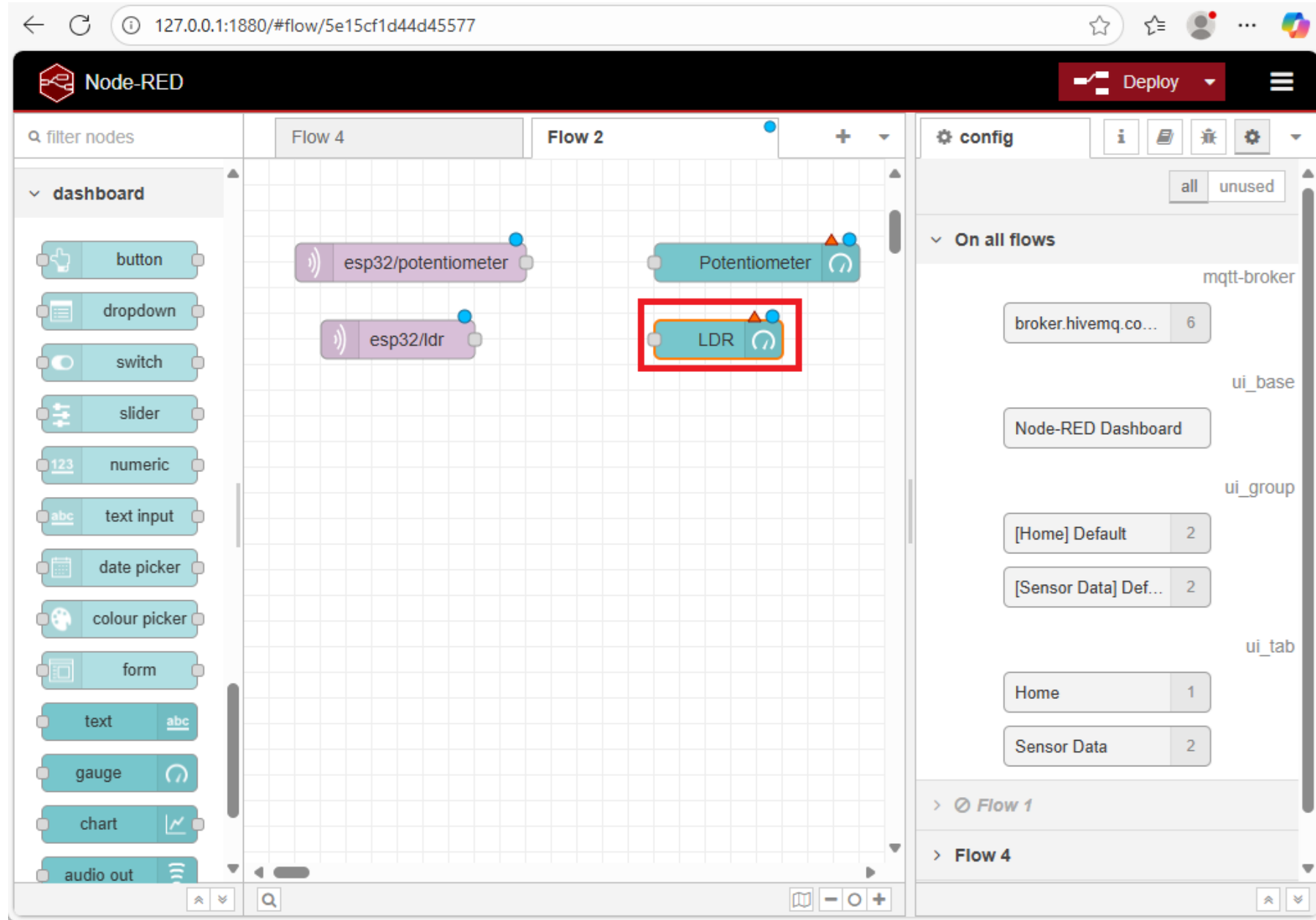


The screenshot shows the Node-RED web interface in a browser window. The address bar displays the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface includes a top navigation bar with the Node-RED logo, a 'Deploy' button, and a hamburger menu. On the left, a 'filter nodes' search bar is present above a list of nodes under the 'dashboard' category, including 'button', 'dropdown', 'switch', 'slider', 'numeric', 'text input', 'date picker', 'colour picker', 'form', 'text', 'gauge', 'chart', and 'audio out'. The central workspace shows a flow with two nodes: 'esp32/potentiometer' and 'esp32/ldr'. A 'gauge' node is highlighted with a red rectangular box. On the right, a 'config' panel is visible, showing settings for 'mqtt-broker', 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default and [Sensor Data] Def...), and 'ui_tab' (Home and Sensor Data). The bottom of the interface features a status bar with zoom controls and a search icon.

- Add the group name, label, and range values in the gauge node's edit window, then select **Done (LDR)**



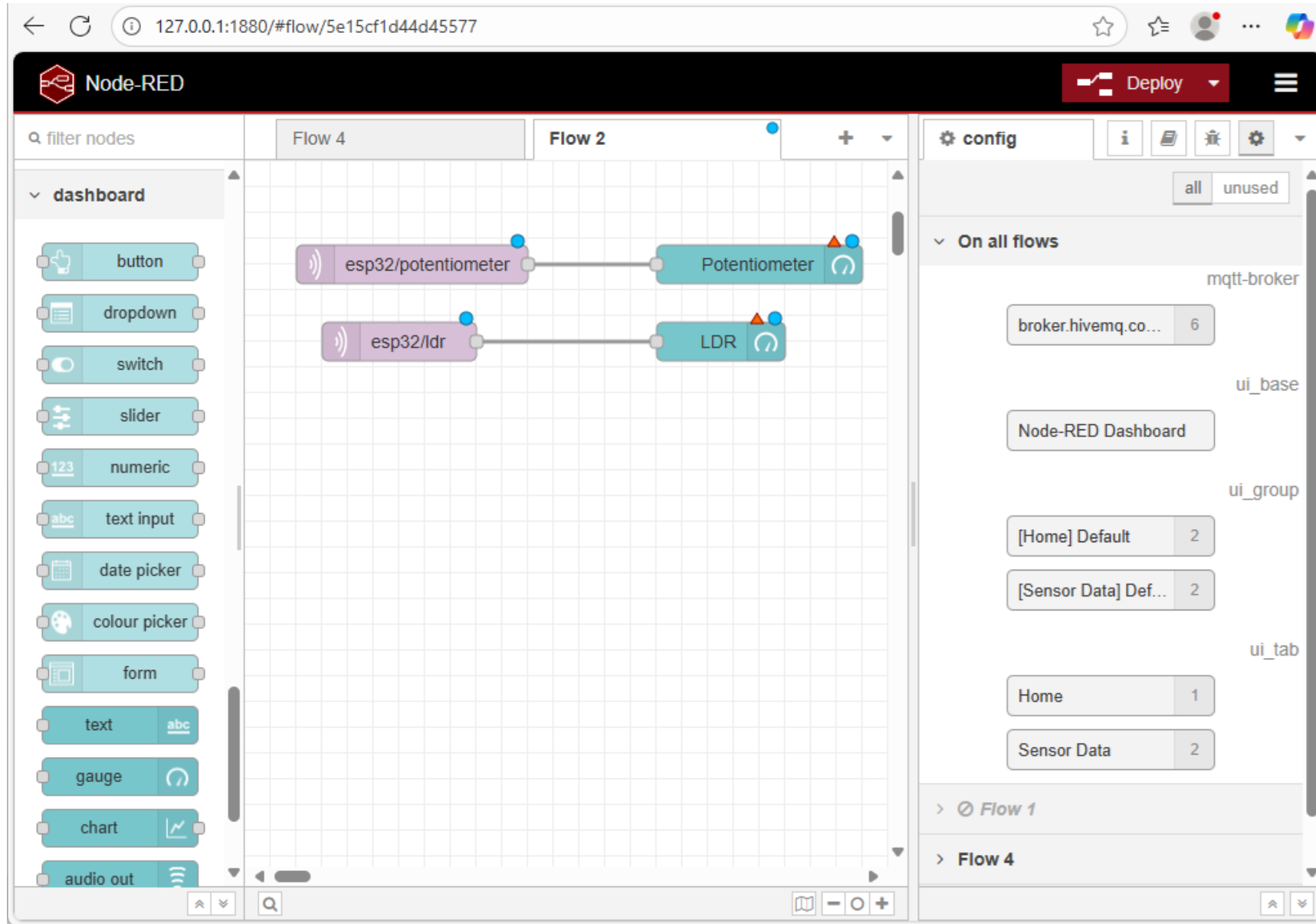
- LDR gauge is configured



The screenshot displays the Node-RED web interface in a browser window. The address bar shows the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface is divided into three main sections:

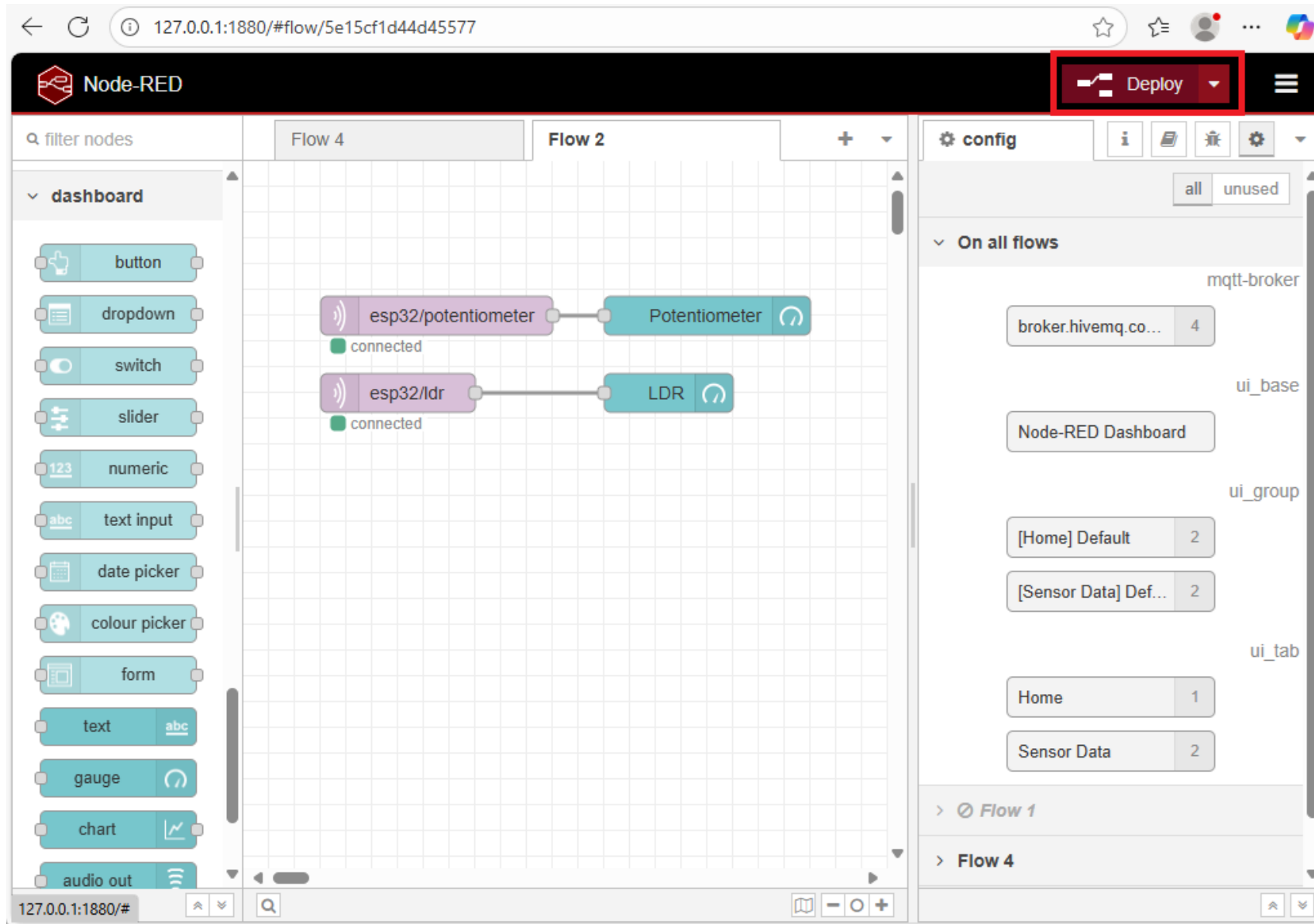
- Left Panel (Palette):** Contains a search bar and a list of nodes under the 'dashboard' category. The 'gauge' node is visible at the bottom of the list.
- Center Panel (Canvas):** Shows a flow diagram with several nodes. A red box highlights the 'LDR' node, which is a gauge node. Other nodes include 'esp32/potentiometer', 'Potentiometer', and 'esp32/ldr'.
- Right Panel (Config):** Displays the configuration for the selected 'LDR' node. It shows settings for 'mqtt-broker' (broker.hivemq.co...), 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default), and 'ui_tab' (Home). The 'Flow 4' tab is selected in the bottom right corner.

- Drag a wire from the MQTT input node (esp32/potentiometer) to the potentiometer gauge.
- Drag a wire from the MQTT input node (esp32/ltr) to the ltr gauge.

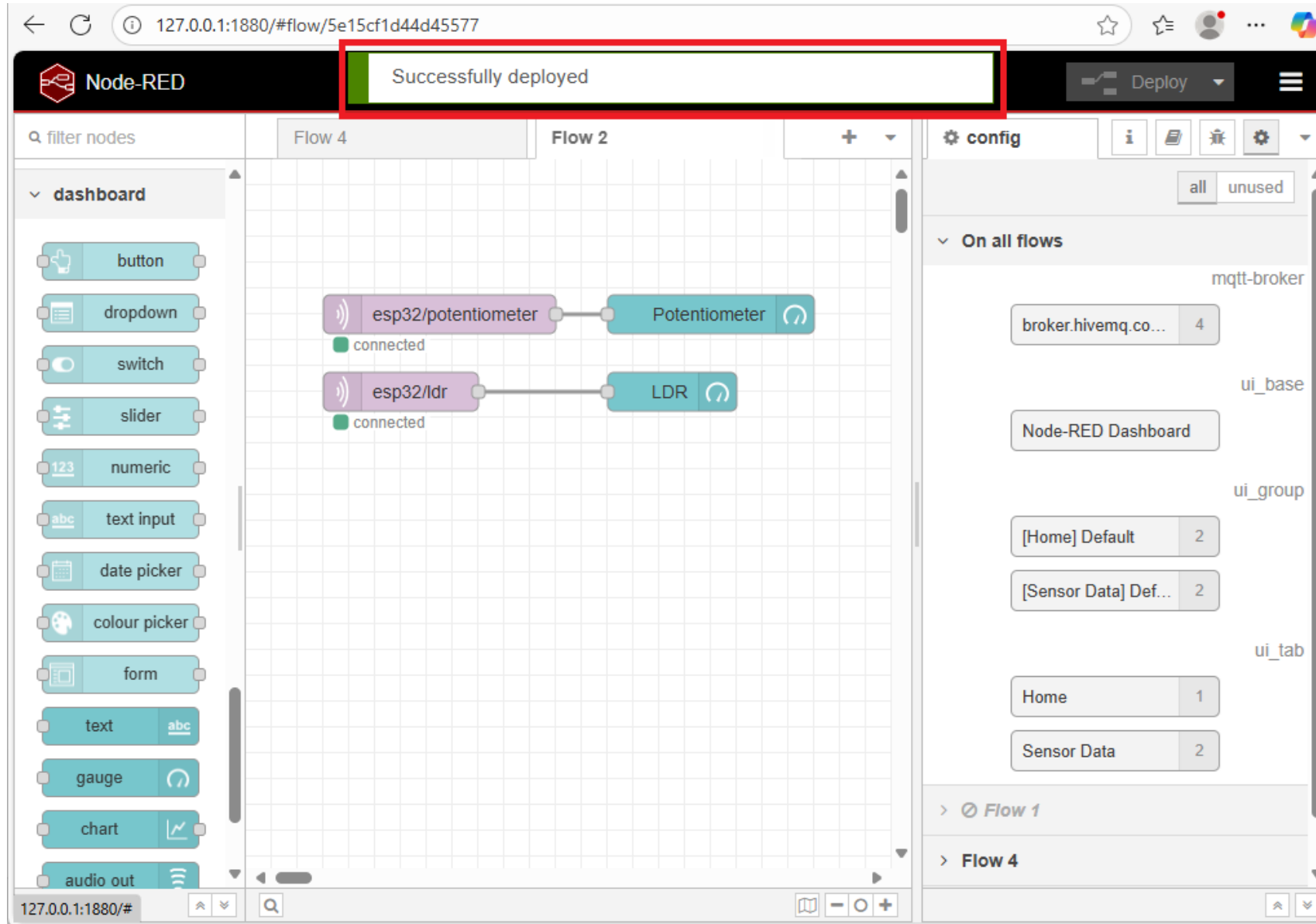


The screenshot shows the Node-RED web interface in a browser window. The address bar displays the URL `127.0.0.1:1880/#flow/5e15cf1d44d45577`. The interface includes a left sidebar with a 'dashboard' menu and a list of nodes: button, dropdown, switch, slider, numeric, text input, date picker, colour picker, form, text, gauge, chart, and audio out. The main workspace shows two flows: 'Flow 4' and 'Flow 2'. 'Flow 2' is active and contains two MQTT input nodes: 'esp32/potentiometer' and 'esp32/ltr'. The 'esp32/potentiometer' node is connected to a 'Potentiometer' gauge node, and the 'esp32/ltr' node is connected to an 'LDR' gauge node. The right sidebar shows the 'config' panel with tabs for 'all' and 'unused'. Under the 'On all flows' section, there are settings for 'mqtt-broker' (broker.hivemq.co...), 'ui_base' (Node-RED Dashboard), 'ui_group' ([Home] Default, [Sensor Data] Def...), and 'ui_tab' (Home, Sensor Data). The bottom of the interface has a search bar and navigation icons.

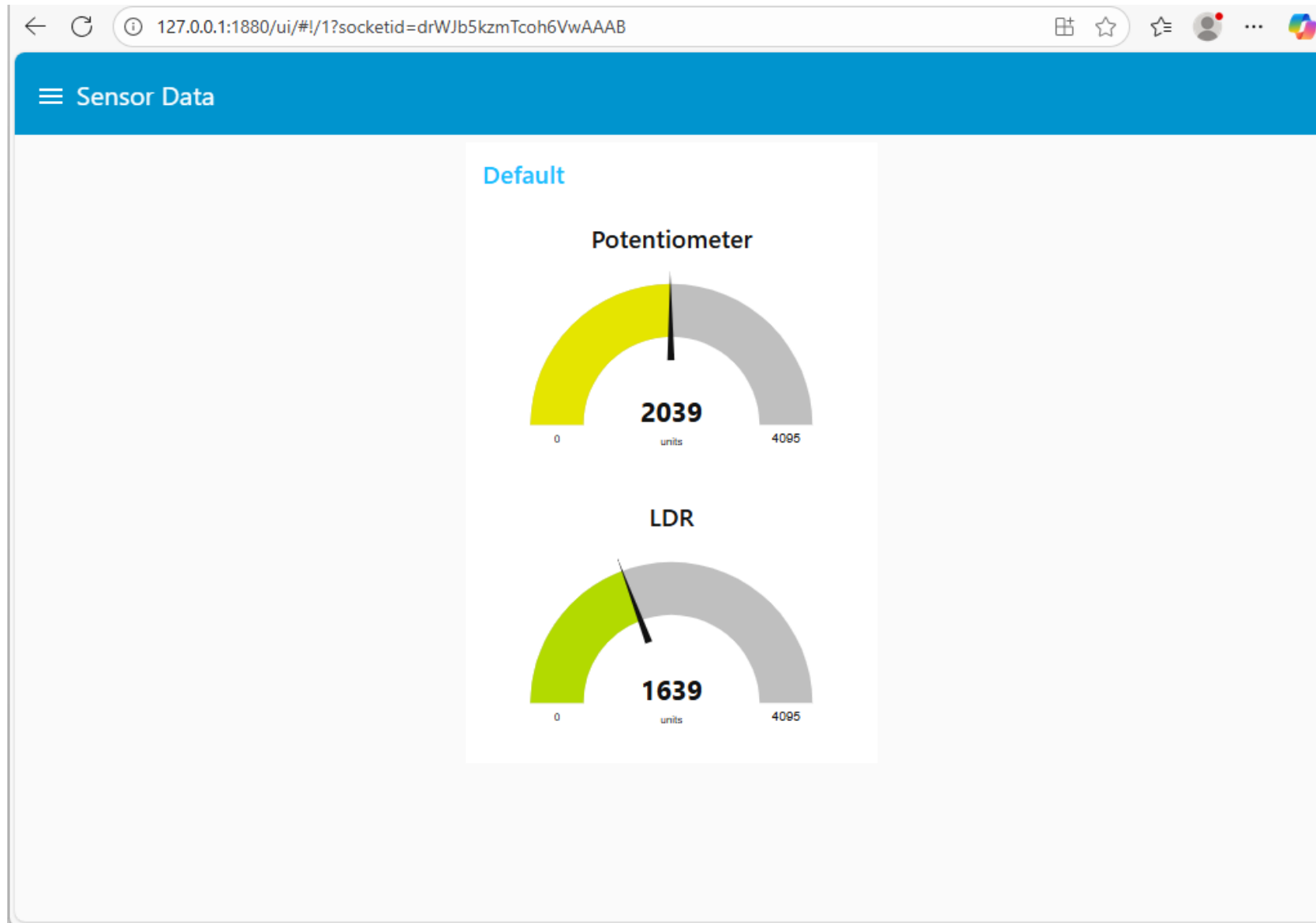
- **Click the Deploy Button:** Locate the red button labeled "Deploy" in the top-right corner of the Node-RED editor.



- **Confirmation:** A successful deployment will usually display a small notification at the top of the screen (e.g., "Successfully deployed").



- **Flow Activation:** Once deployed, the logic you wired—the Dashboard Gauges publishing LDR and Potentiometer values—becomes active



Thank You